

软件工程基础

刘钦

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

知识点

- 软件的定义 (P52)
- 软件的类型 (P50)
- 软件工程的定义 (P56)
- 工程学科的发展规律 (P59)
- 工程师的理解 (P60, P61, P62)
- 软件开发活动和产物 (P68, P69)
- 软件开发的角色分工 (P70-P62)

知识点

- 项目的定义 (P86)
- 项目管理的目标 (P87)
- 过程组与活动的区别 (P88)
- 团队的特征 (P91)
- 团队的结构 (P92) 三者的区别
- 团队建设的措施 (P96)
- 质量模型的定义 (P101-P103)
- 质量保证活动 (P104)
- 评审的过程 (P106-P107)
- Measure、Measurement、Metrics的区别 (P108)
- 配置管理的定义 (P111)
- 配置项的定义 (P112)
- 基线的定义 (P113)
- 配置管理的活动 (P114)
- 分支管理常见策略 (P120)
- 变更控制过程 (P126-P127)

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

Software

What is software?

- Programming Language?
- Program?
- Contrast to hardware?
- ...

Information

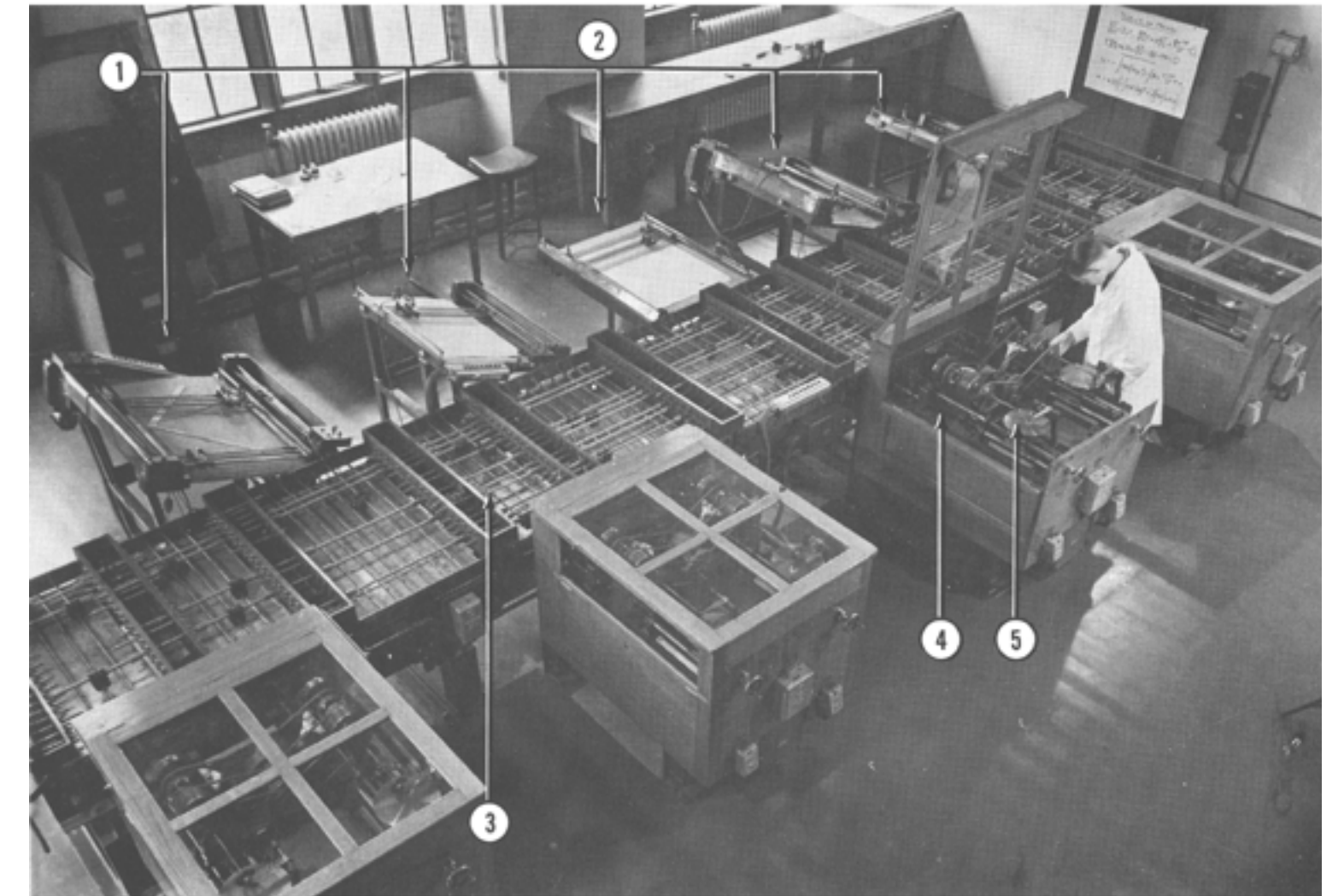
- 信息的记录
 - 泥板、莎草纸、龟壳、石头、竹子、纸、磁带、磁盘、光盘、硬盘
- 信息的交流
 - 口头相传、信号、手语、电报、电话、卫星、互联网
- 信息的存储
 - 书籍、图书分类、数据库、数据仓库、数据湖

变革

- Information => Computing => Digital Computer => Software=>AI ?

Foundation of Digital Computing - 30s

Hardware



- 微分计算器 Vannervar Bush
- 继电器式计算机 Konard Zuse



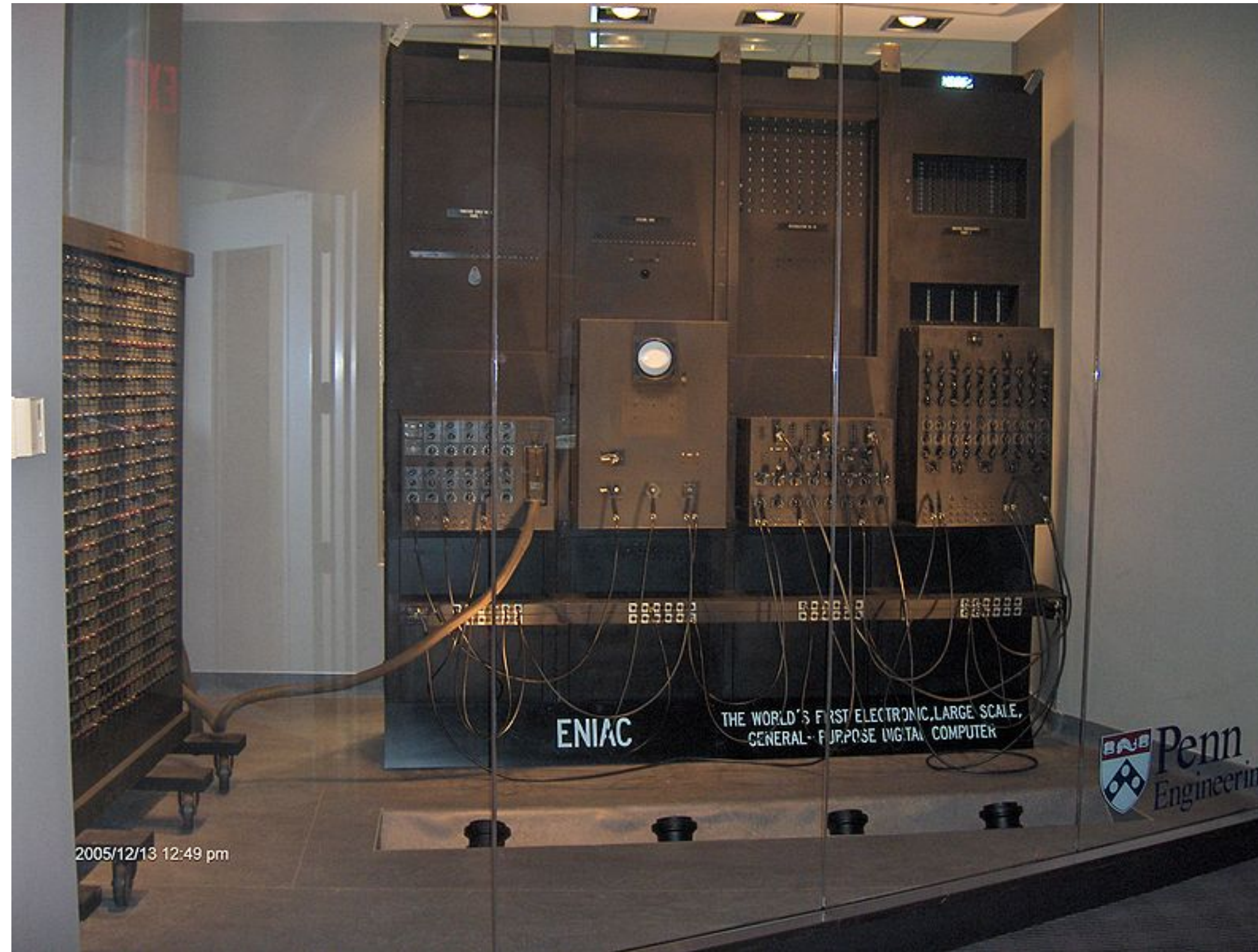
Theory

- 《论可计算数及其在判定问题上的应用》 Alan Turing 1936
- 《电子继电器可以实现布尔符号逻辑》 Claude Elwood Shannon 1937

**Software is one part of
hardware - 40s**

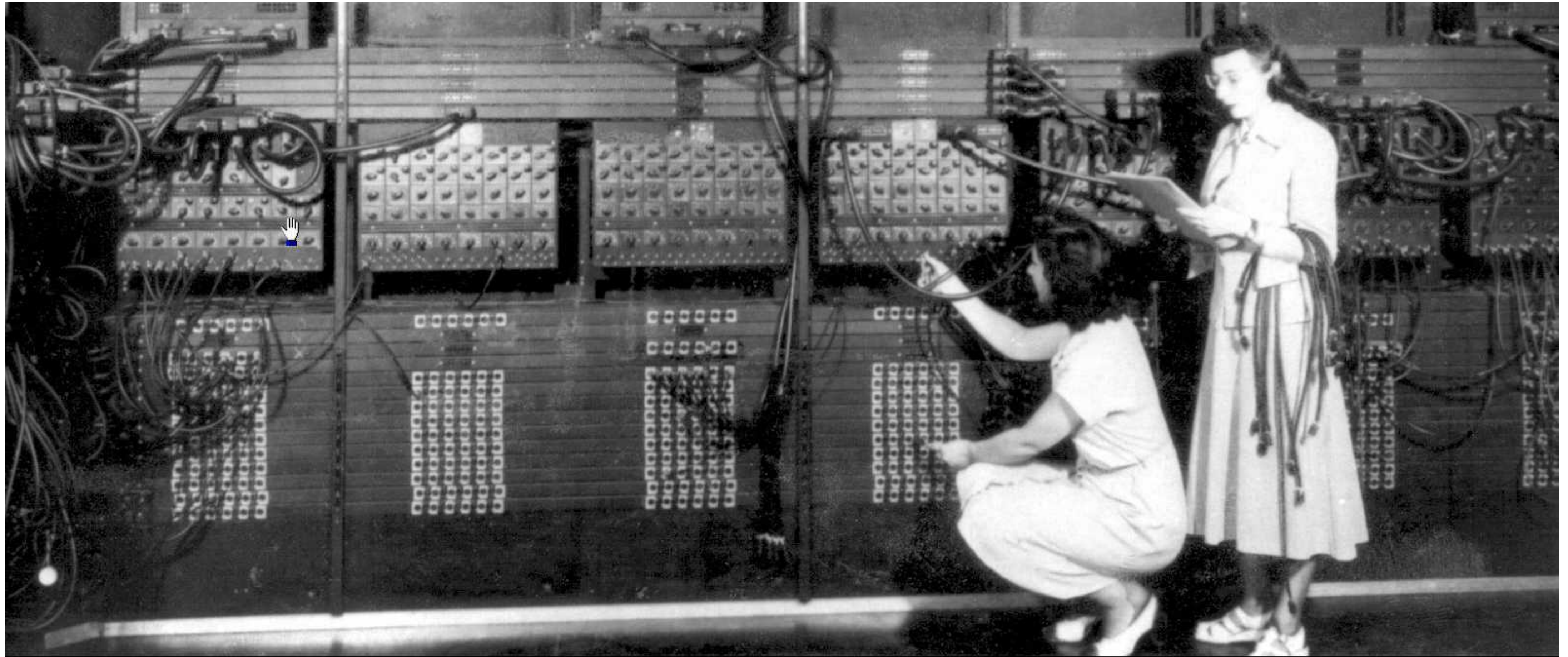
Hardware

- ABC原型计算机 John Vincent Atanasoff & Clifford Berry 1939-1942
- Harvard Mark 1 IBM实验室 1939-1944
- ENIAC 1943-1946
- EDVAC 1944-1949



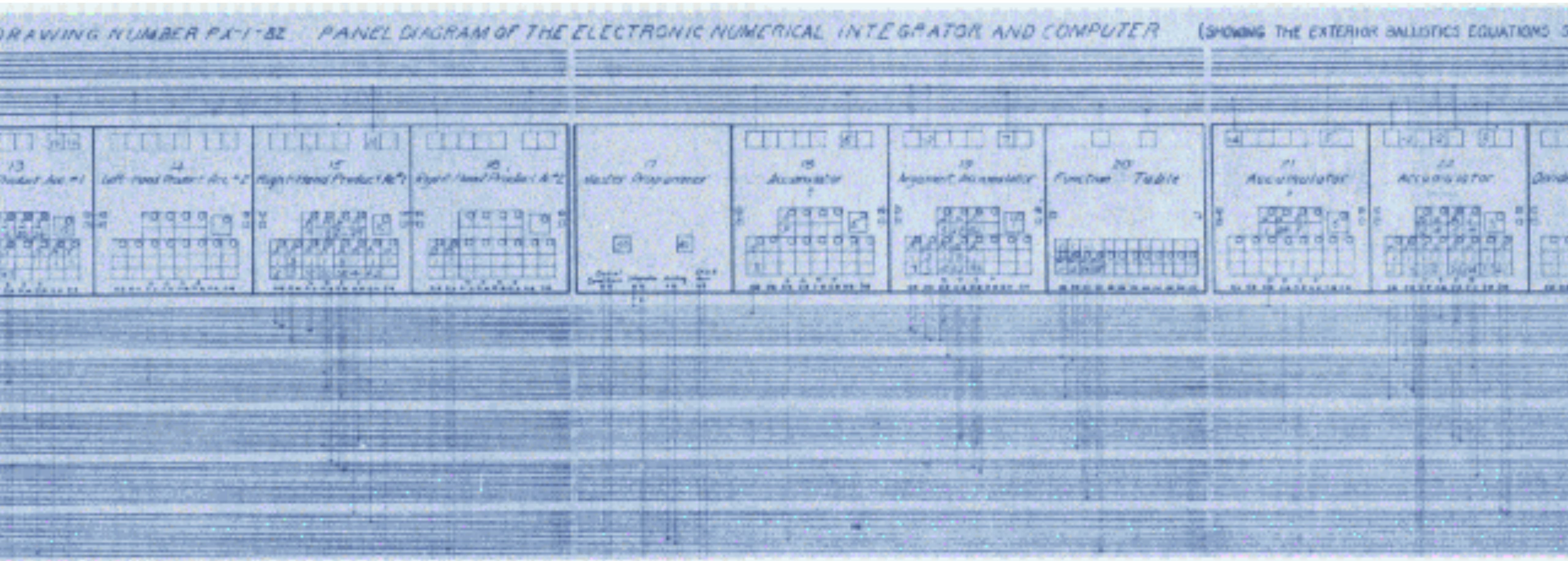
ENIAC

Four ENIAC panels and one of its three function tables, on display at the School of Engineering and Applied Science at the University of Pennsylvania



Programming the ENIAC

Two women set the switches on one of ENIAC's function tables at the Moore School of Electrical Engineering.



Programming chart

ENIAC programming chart representing the wiring to set up an exterior ballistics equation

Steps of programming

- The task of taking a problem and **mapping** it onto the machine was complex, and usually took weeks.
- After the program was figured out on paper, the process of getting the program "**into**" ENIAC by manipulating its switches and cables took additional days.
- This was followed by a period of **verification** and **debugging**, aided by the ability to "single step" the machine.

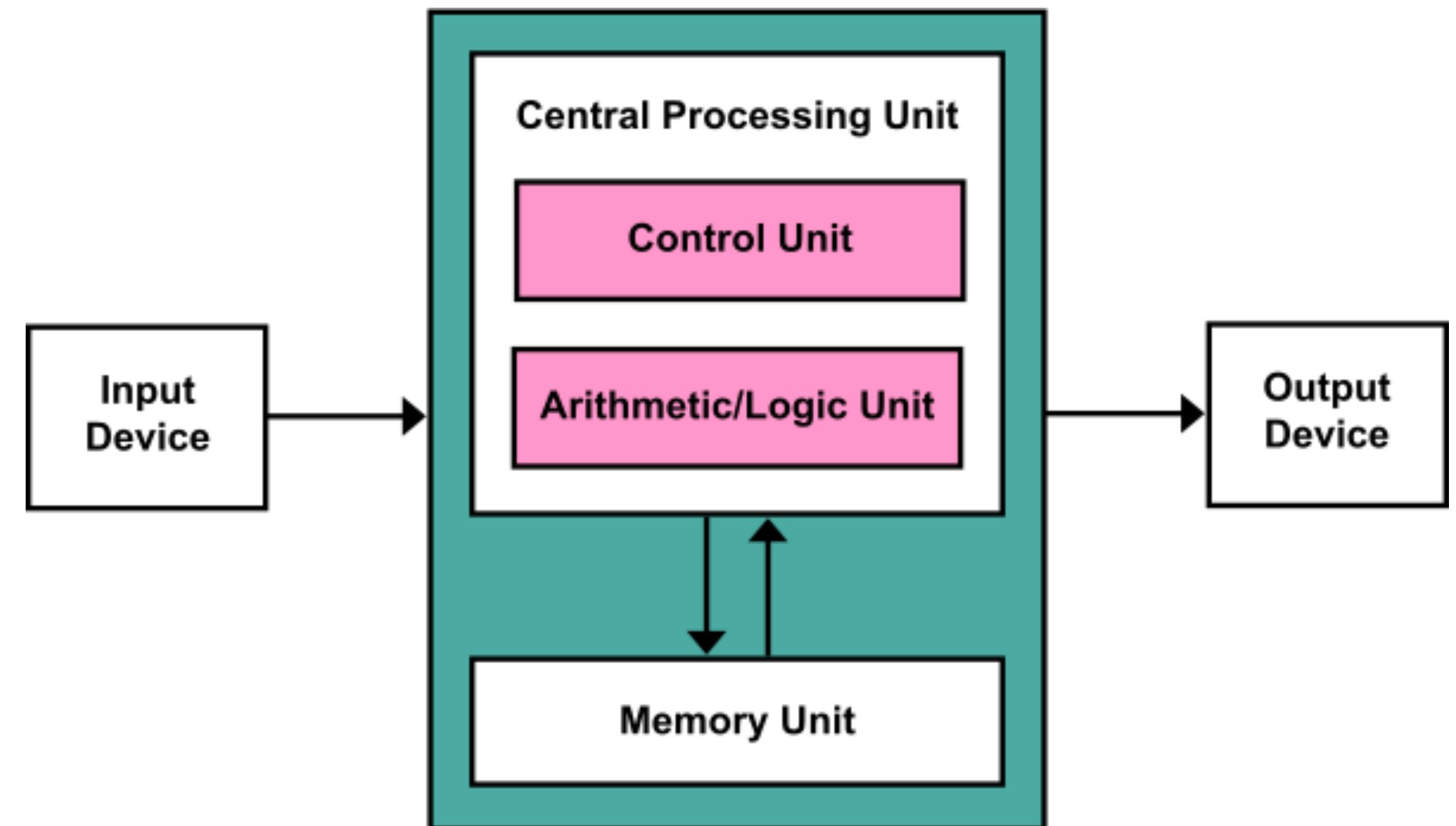
EDVAC

- EDVAC (Electronic Discrete Variable Automatic Computer) was one of the earliest electronic computers. Unlike its predecessor the ENIAC, it was binary rather than decimal, and was a stored program computer.
- Eckert and Mauchly and the other ENIAC designers were joined by [John von Neumann](#) in a consulting role; von Neumann summarized and discussed logical design developments in the 1945 First Draft of a Report on the EDVAC.



Von Neumann architecture

- The stored-program concept



应用领域

- 科学 (75)
- 国防和军事 (100)
 - 鱼雷瞄准
 - 飞机投弹
- 平民政府 (10)
- 系统和中间件 (5)
- 商用 (5)

**Software Engineering is Same
as Hardware Engineering - 50s**

Commercial Computer

- Ferranti Mark I
- UNIVAC I
- LEO I
- IBM 701
- IBM 650

Programming Language

- 1955 FORTRAN
- 1958 LISP
- 1959 COBAL

Naming

- In a 1958 article in American Mathematical Monthly, [John W. Tukey](#) became the first person to define the programs on which electronic calculators ran.

Software is not Hardware - 60s

变革

- ASCII美国信息交换标准码
- ATM
- IBM信息管理系统IMS应用于阿波罗航天器
- 软件咨询业务
- IBM S/360
- 信用卡
- DEC PDP-1小型机

软件的特性

- 软件与现实世界关系更加密切，对需求的规格化更加困难
- 软件比硬件容易修改的多，并且不需要昂贵的生产线复制产品
- 软件没有损耗
- 软件不可见

**Program = Algorithm + Data
Structure - 70s ~ 80s**

Program

- Algorithms + Data Structures = Programs is a 1976 book written by [Niklaus Wirth](#) covering some of the fundamental topics of computer programming, particularly that algorithms and data structures are inherently related.
- For example, if one has a sorted list one will use a search algorithm optimal for sorted lists.

**Software Development is much more
complicated than Programming**
- 90s ~ 2010s

应用

- 1990-1999 万维网的发展
- 2000-2009 社交网络 移动电商

软件工作量随应用程序规模变化的情况

规模 (功能点)	规模 (KLOC)	编码 (%)	文档编写 (%)	缺陷清除 (%)	管理和支持 (%)
1	0.1	70	5	15	10
10	1	65	7	17	11
100	10	54	15	20	11
1000	100	30	26	30	14
10, 000	1000	18	31	35	16

- 源自[Jones1996]

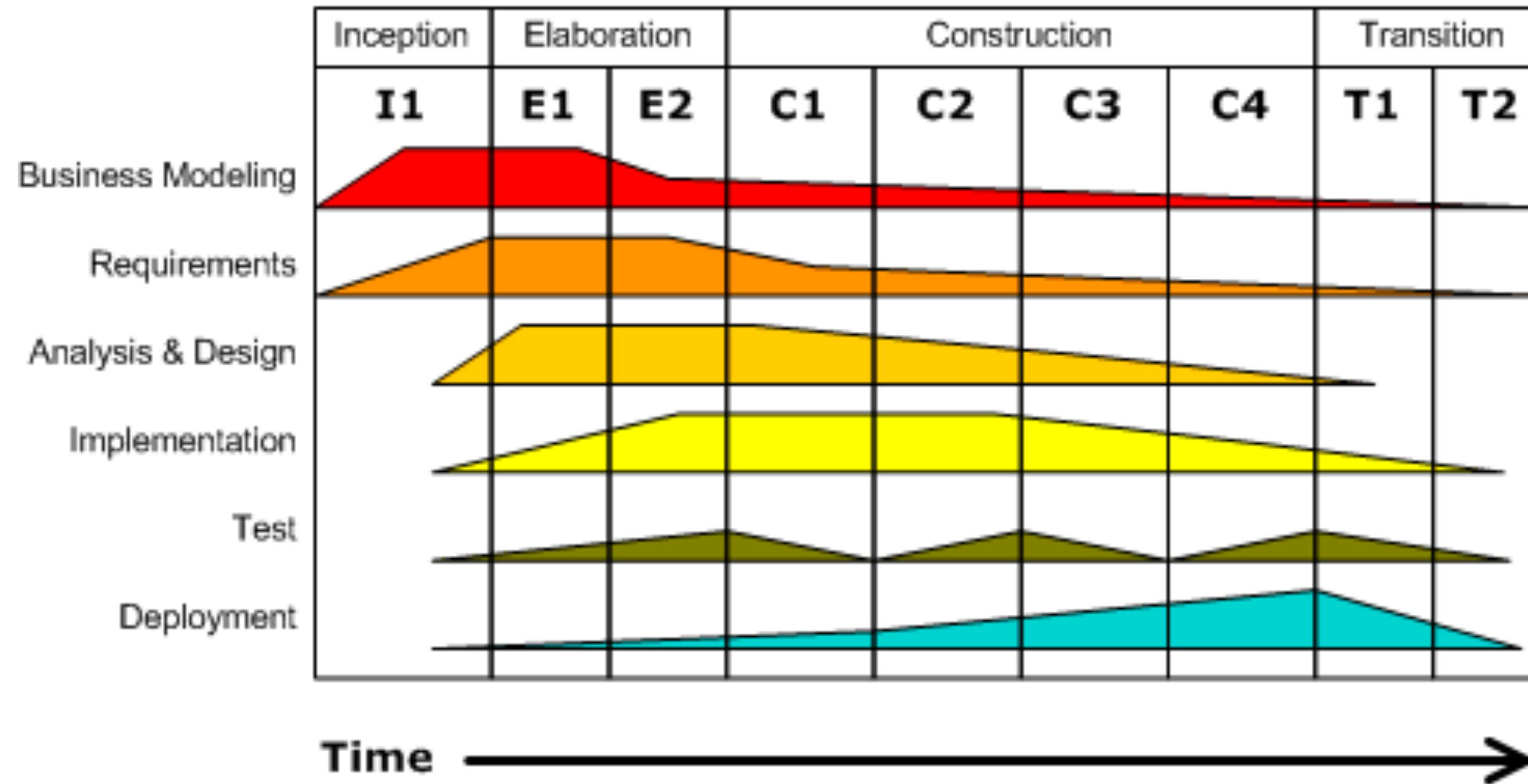
大型软件系统（规模为100万LOC）的成本因素排序

排名	工作内容
1	缺陷清除（审查、测试、发现并修复Bug）
2	纸质文档编写（计划、规格说明、用户手册）
3	会议和交流（顾客、团队成员、经理）
4	编程或编码
5	项目管理
6	变更控制

- 源自[Jones2007]

Iterative Development

Business value is delivered incrementally in time-boxed cross-discipline iterations.



RUP

Cloud Native
2010s - now

软件的应用

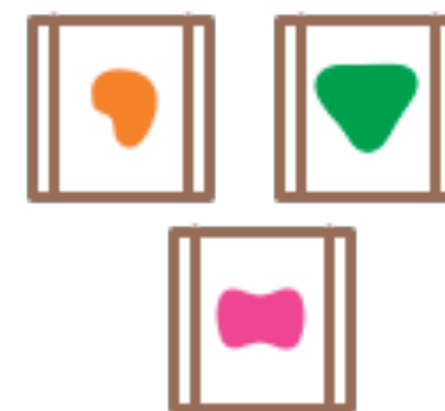
- 云 (docker、kubernetes、istio)
- 众 (众筹、众包)
- 大数据 (Spark、hadoop、DolphinScheduler、ClickHouse、DataSophon、SeaTunnel、Flink、kafka、Elasticsearch)
- AI (AlphaGo, AlphaCode, ChatGPT, Sora)
- 短视频、Web3.0

云原生

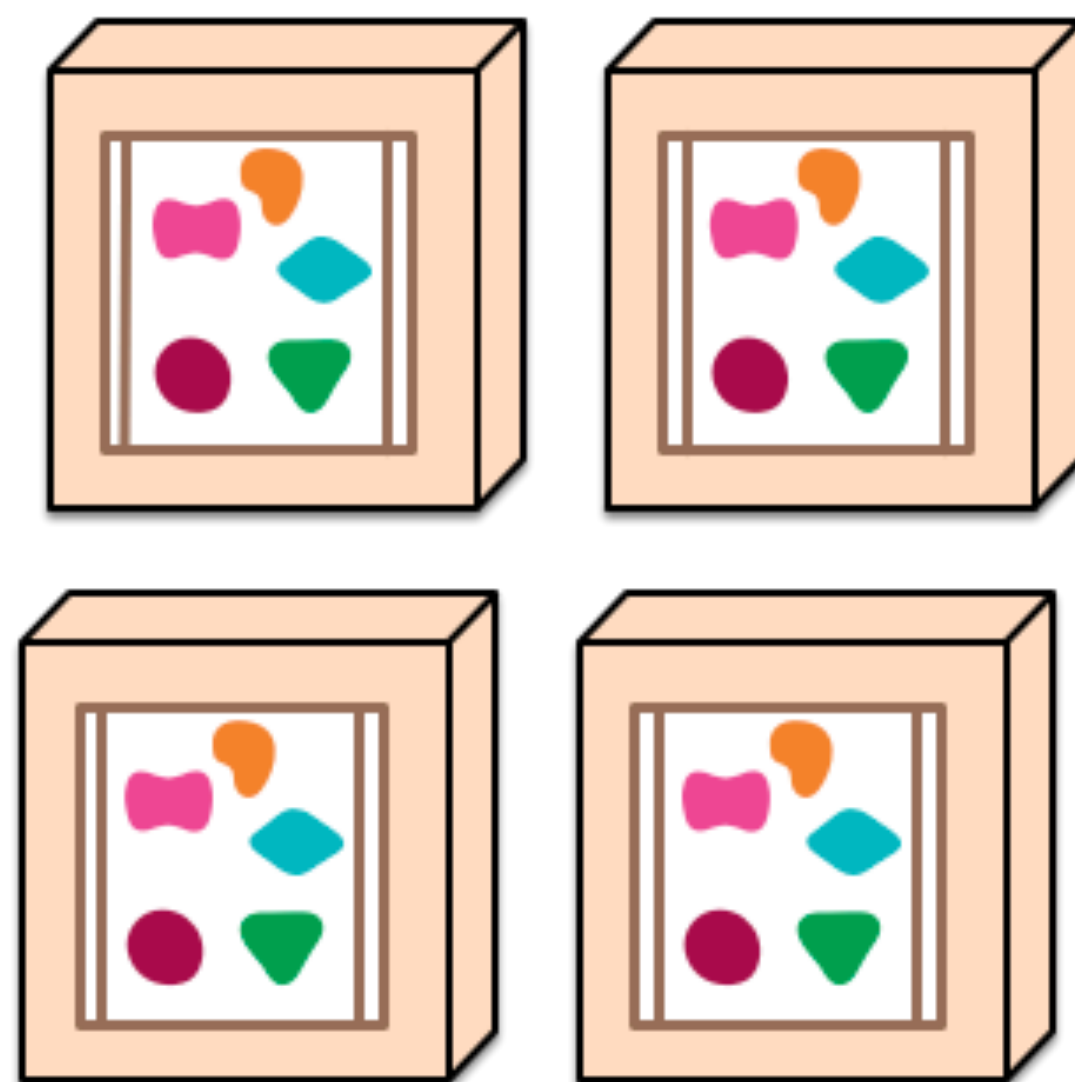
一个单体应用程序把它所有的功能放在一个单一进程中...



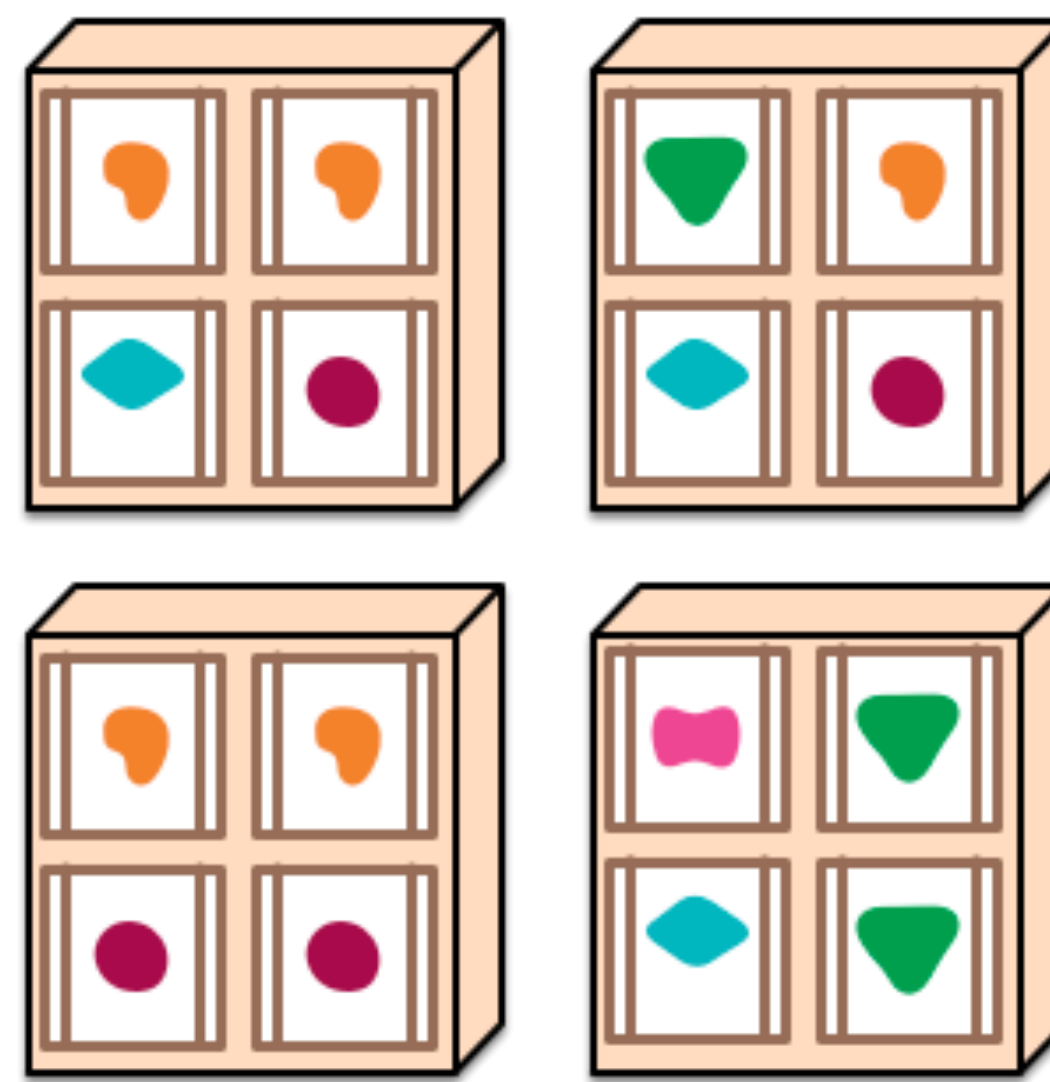
一个微服务架构把每个功能元素放进一个独立的服务中...



...并且通过在多个服务器上复制这个单体进行扩展

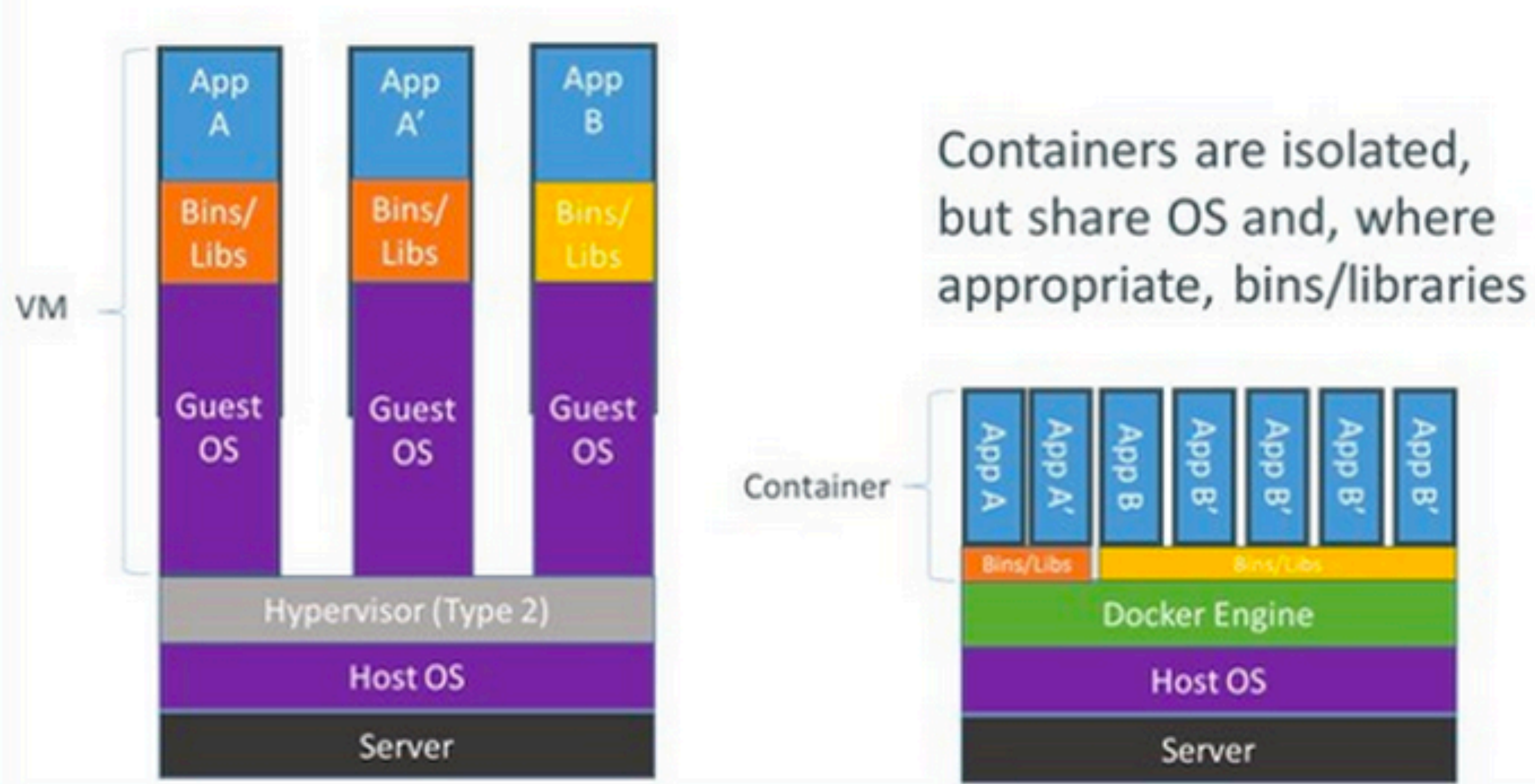


...并且通过跨服务器分发这些服务进行扩展，只在需要时才复制。

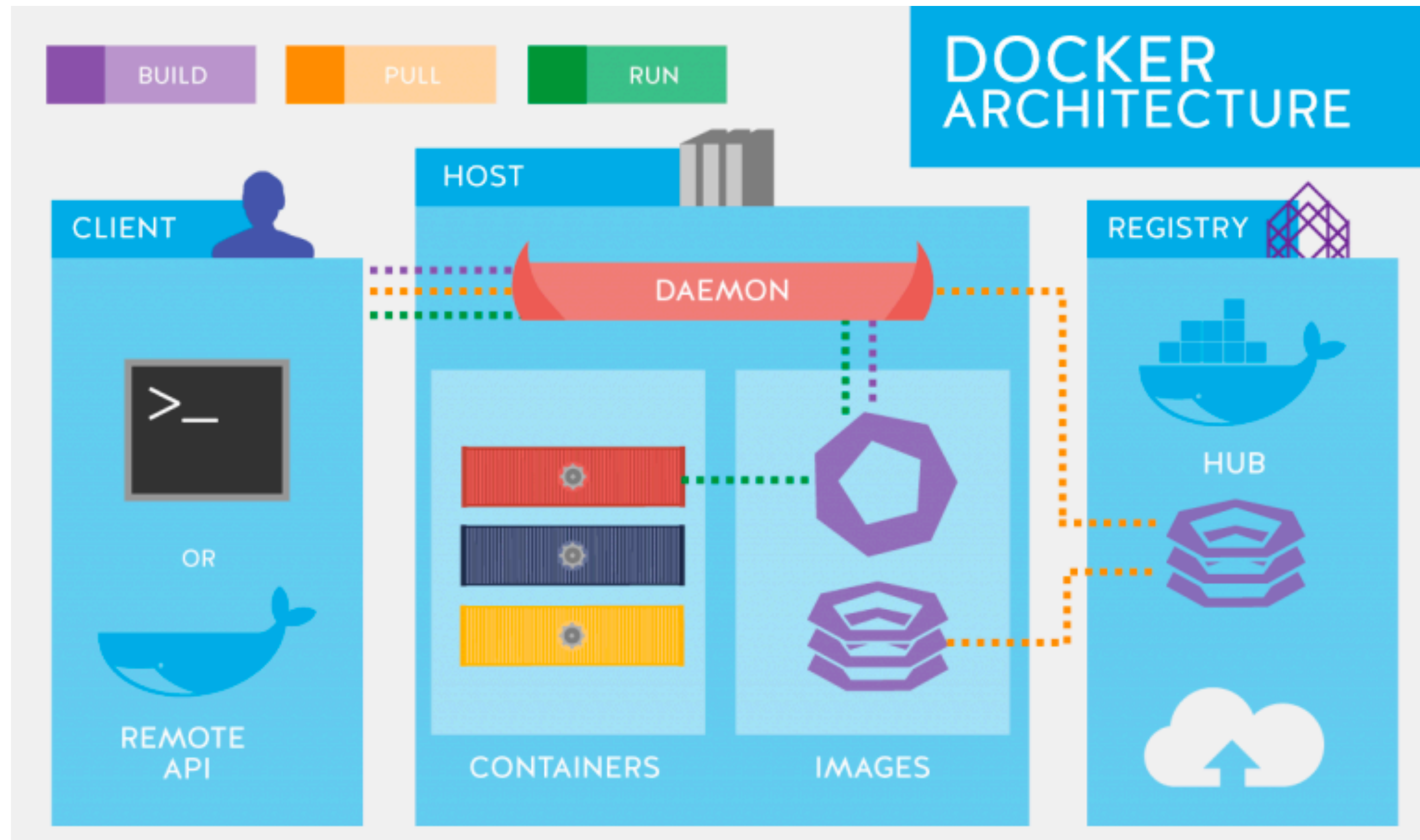


单体和微服务

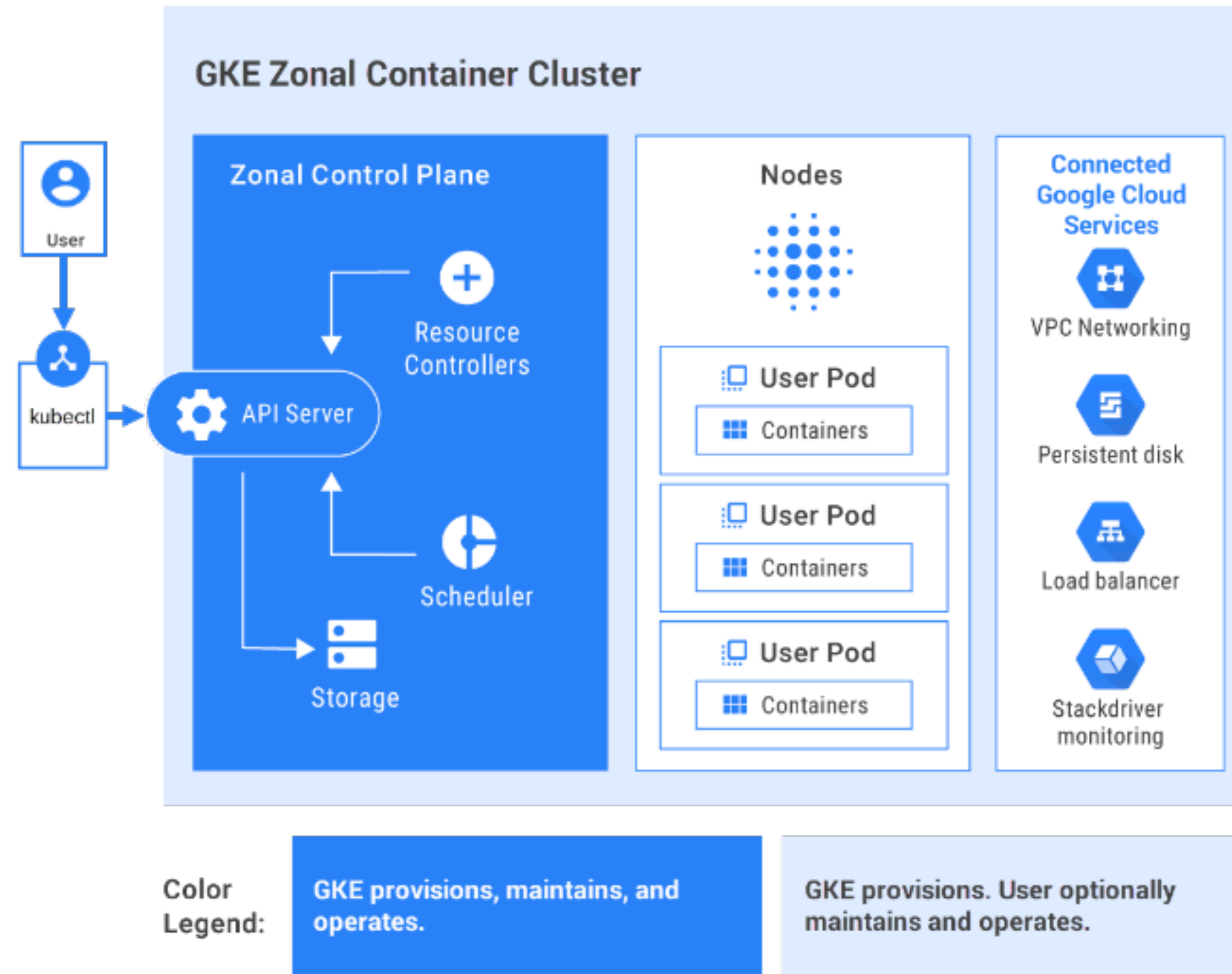
Containers vs. VMs



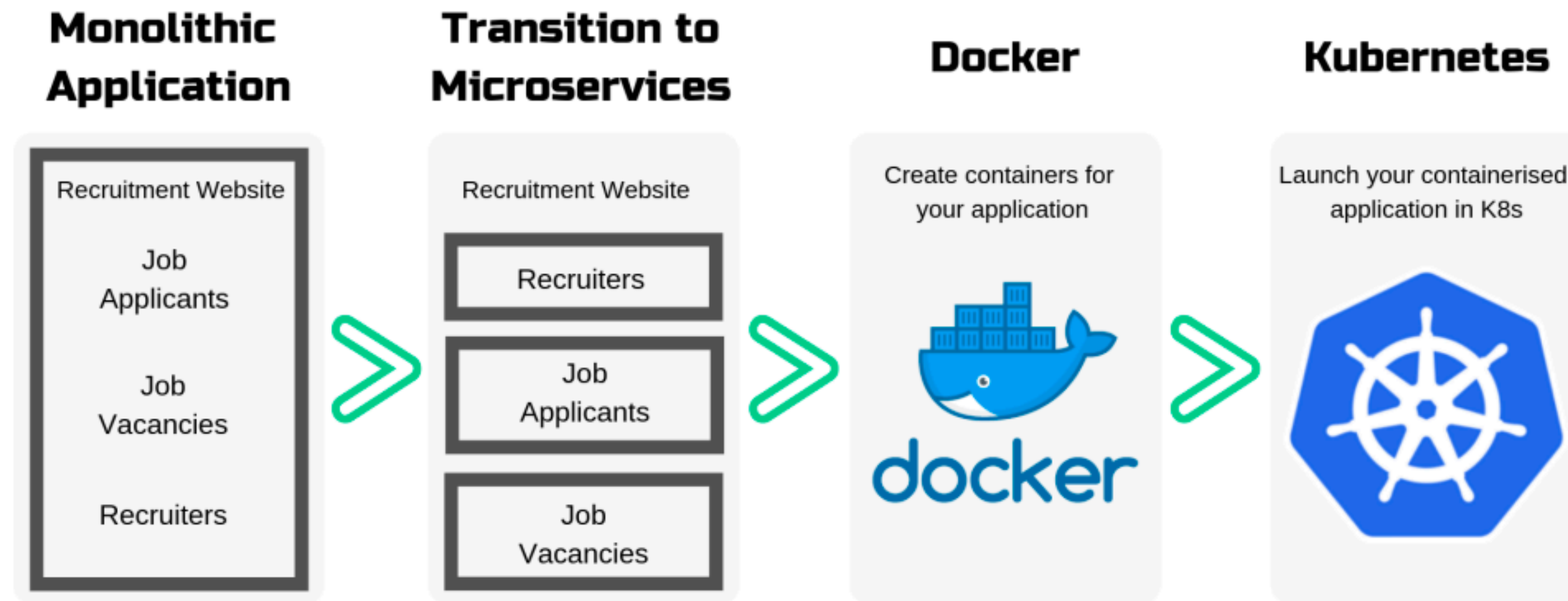
Containers vs VMs



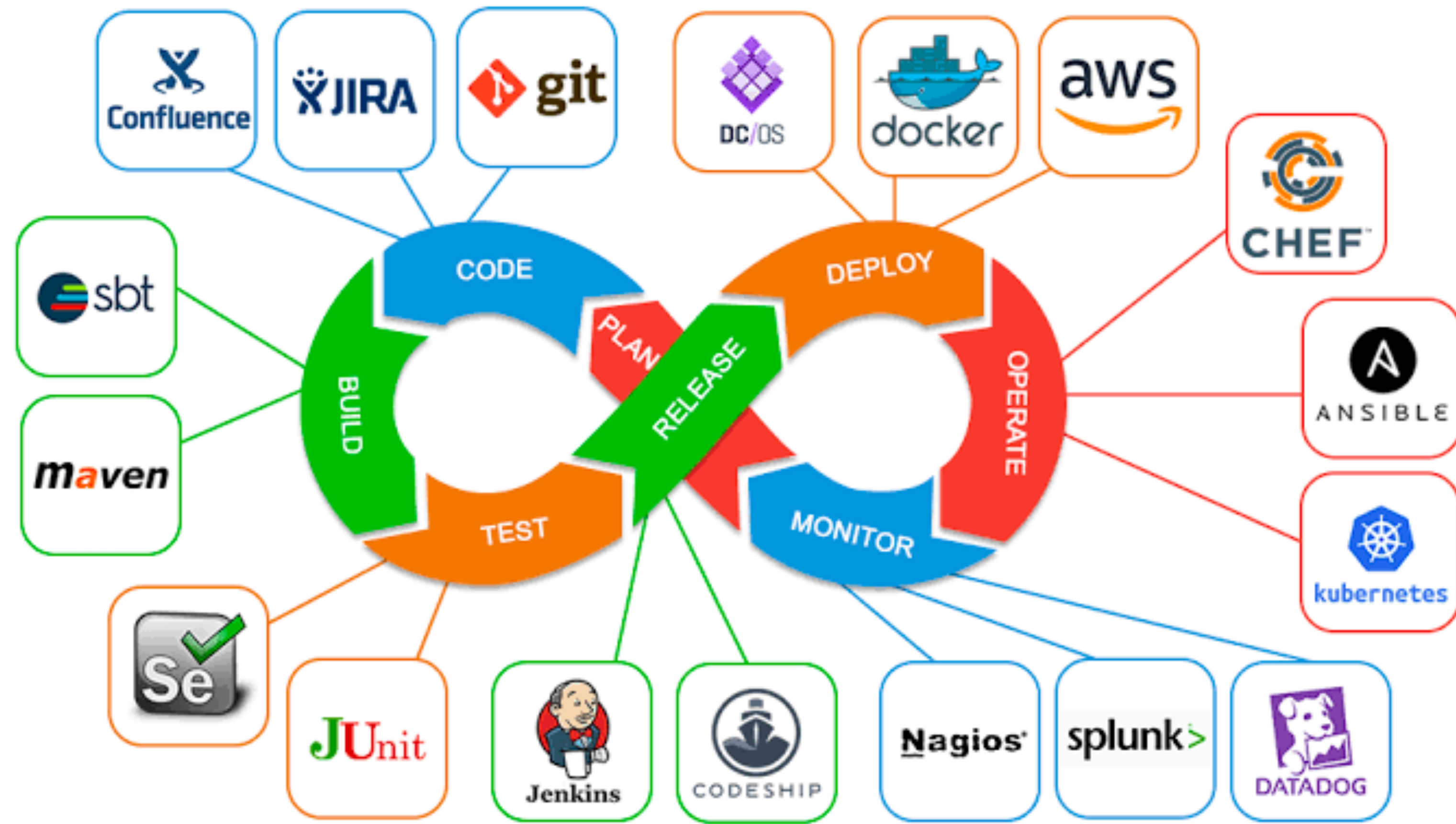
Docker Architecture



Kubernetes Architecture

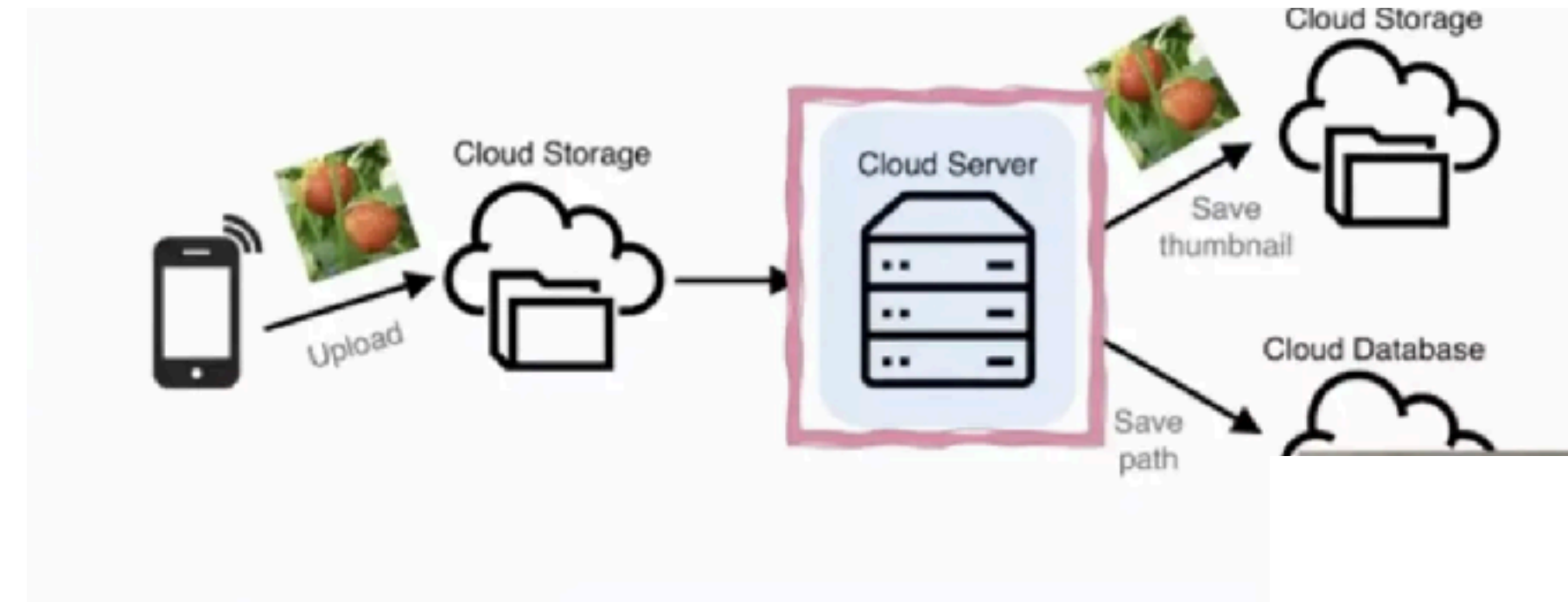


一个微服务案例

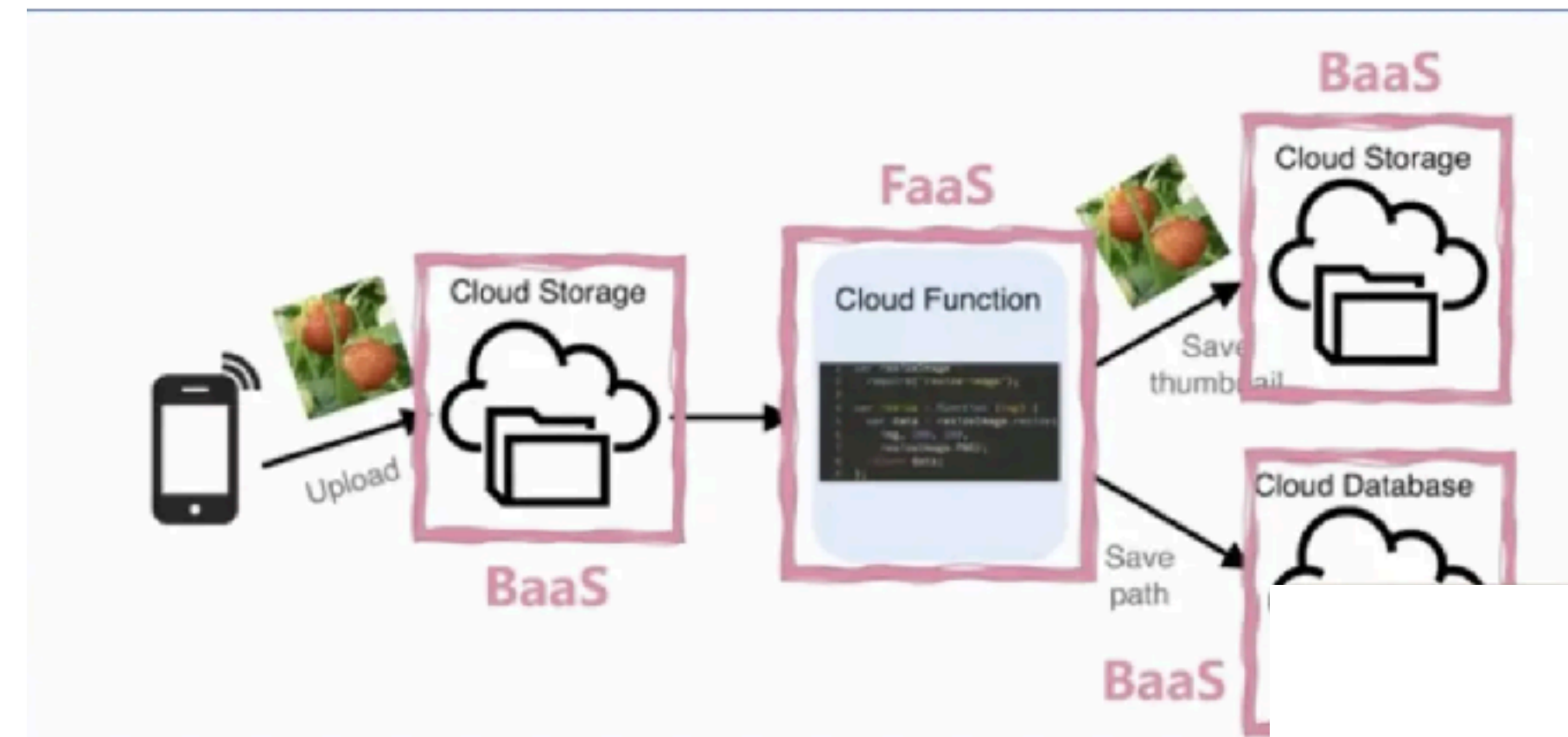


DevOps is as much about culture, as it is about the toolchain!

传统云服务

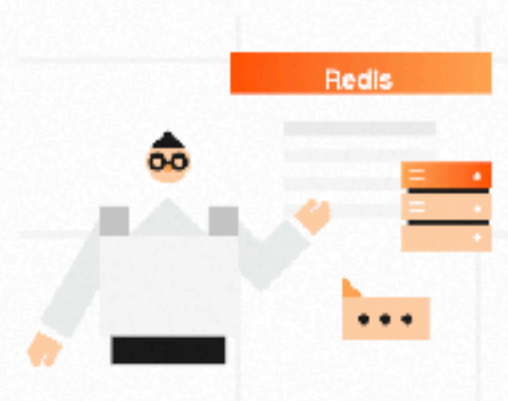


Serverless



Serverless= FaaS + BaaS

我的资源



你还没有云数据库MongoDB实例

仅需3步，快速使用Mongo服务，开箱即用，运维简单。

1

创建实例

2

设置白名单

3

连接实例

创建副本集实例

创建分片集实例

帮助文档

功能亮点

云数据库 MongoDB 版 - 7.0 版本发布 **HOT**

新功能

规格

云数据库 MongoDB 版发布了MongoDB 7.0版本，该版本新增了可查询加密、分片元数据一致性校验、查询采样、分析分片键以及自动合并等新特性，并对分片、时序集合、聚合以及数据库安全性等方面进行了优化。副本集和分片集群实例均已支持MongoDB 7.0版。[查看详情>>>](#)

高频备份

新功能

为了保证您的数据安全，云数据库 MongoDB 版提供了高频备份功能，您可以根据需求设置分钟级别或者小时级别的高频备份。本文介绍如何设置高频备份。[查看详情>>>](#)

云盘加密

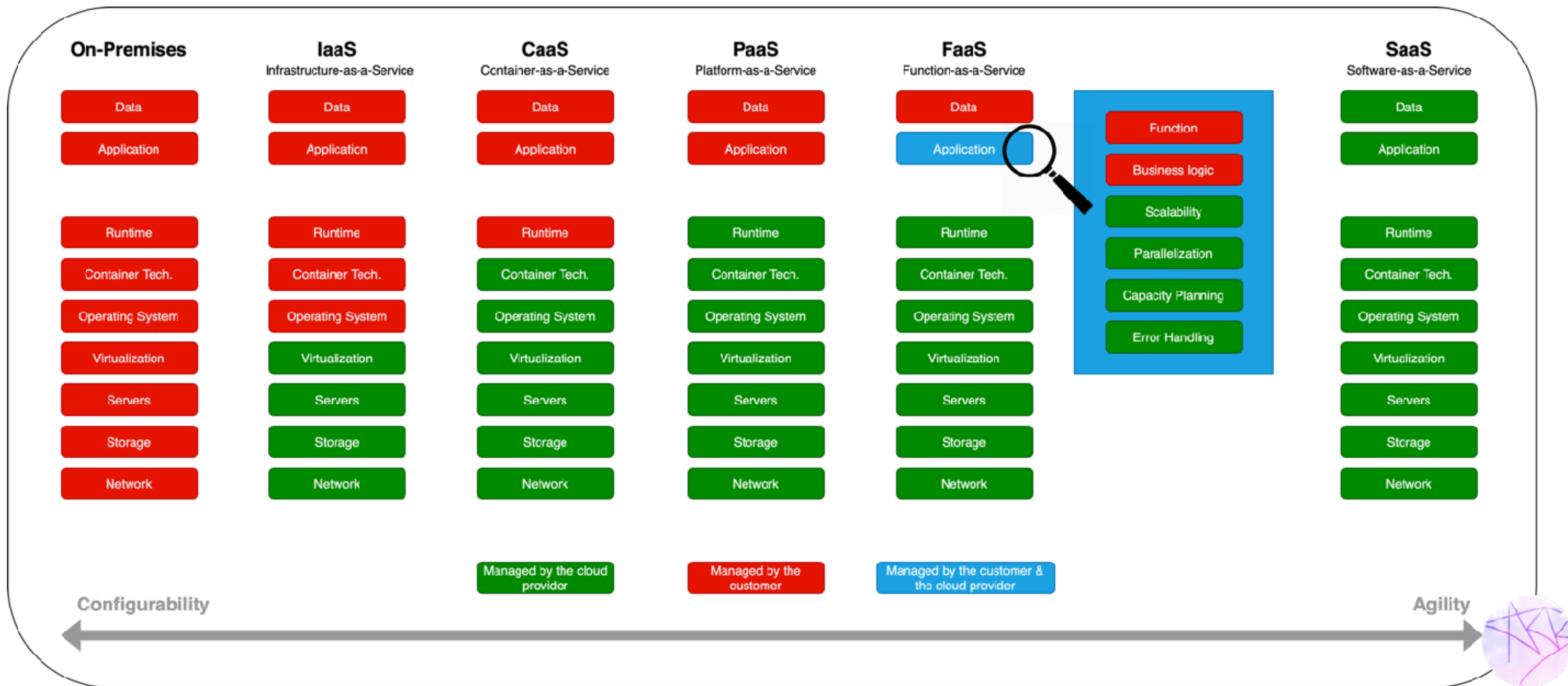
新功能

云盘加密可以基于块存储对整个数据盘进行加密，同时该云盘生成的快照及这些快照创建的云盘将自动延续加密属性，可以最大限度保护您的数据安全。云盘加密不会影响您的业务，您无需更改任何应用程序。本文介绍如何开通云盘加密。[查看详情>>>](#)

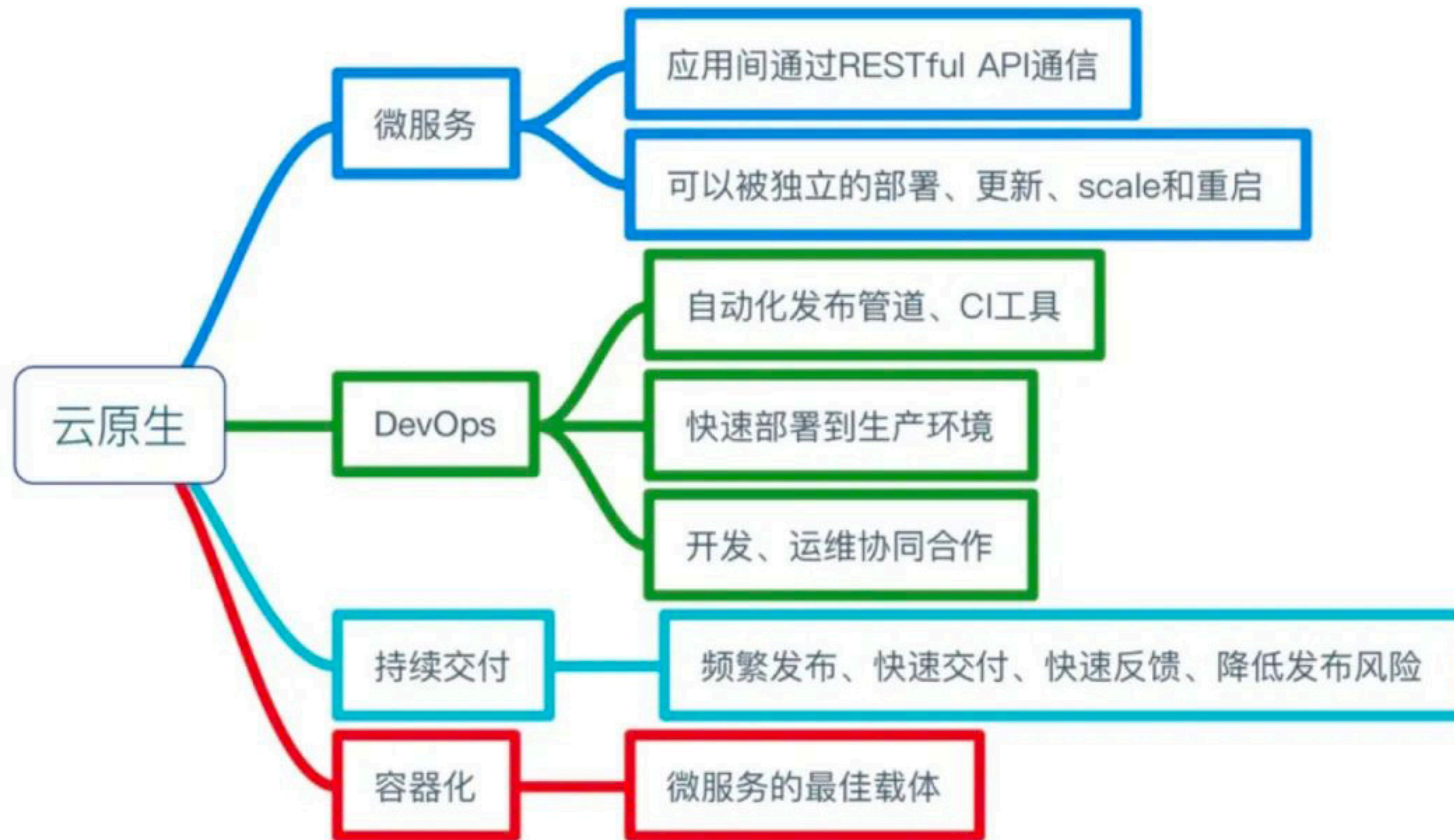
阿里云数据库Baas服务示例



阿里云函数计算示例



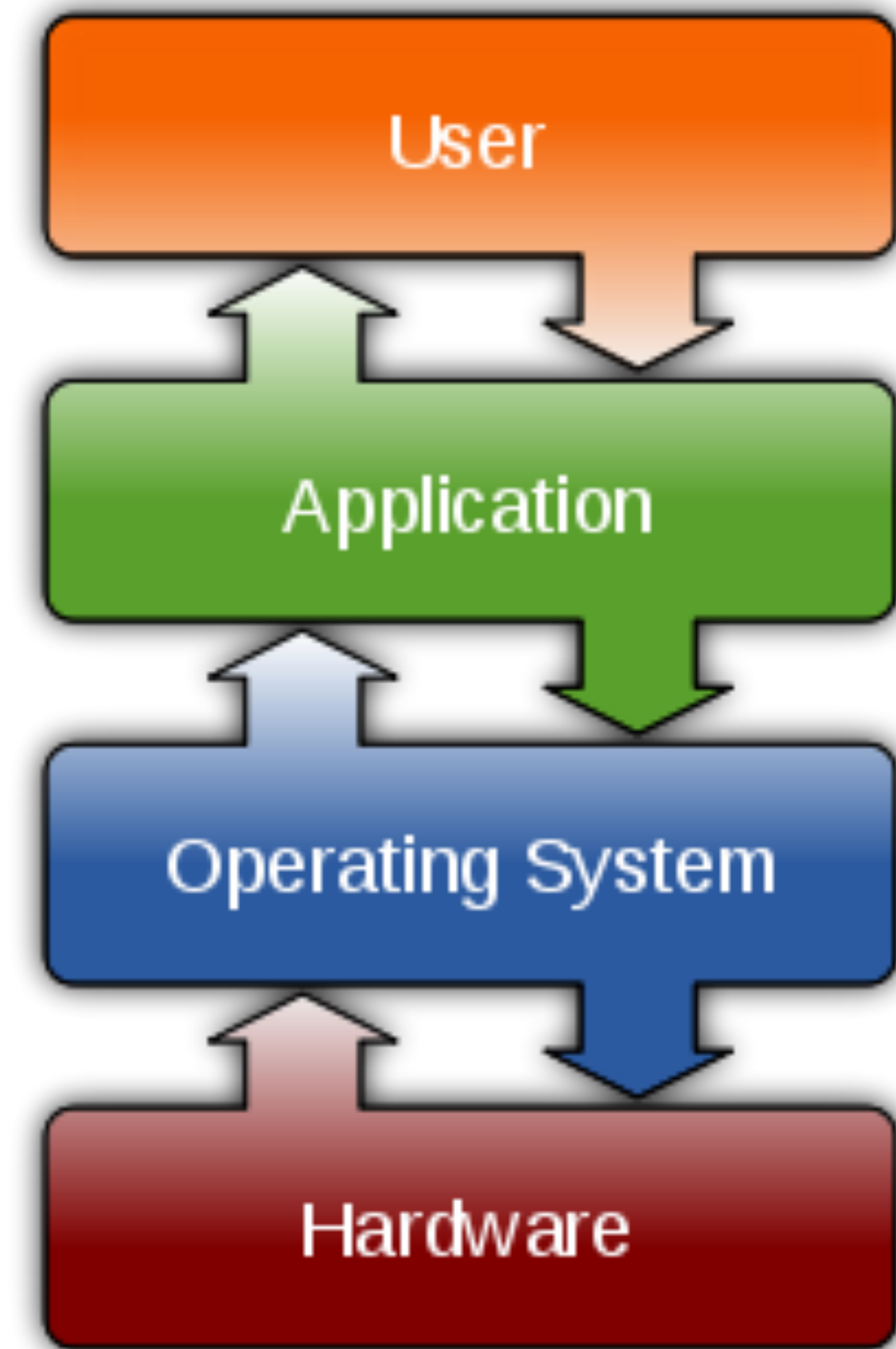
IaaS to FaaS



云原生内涵

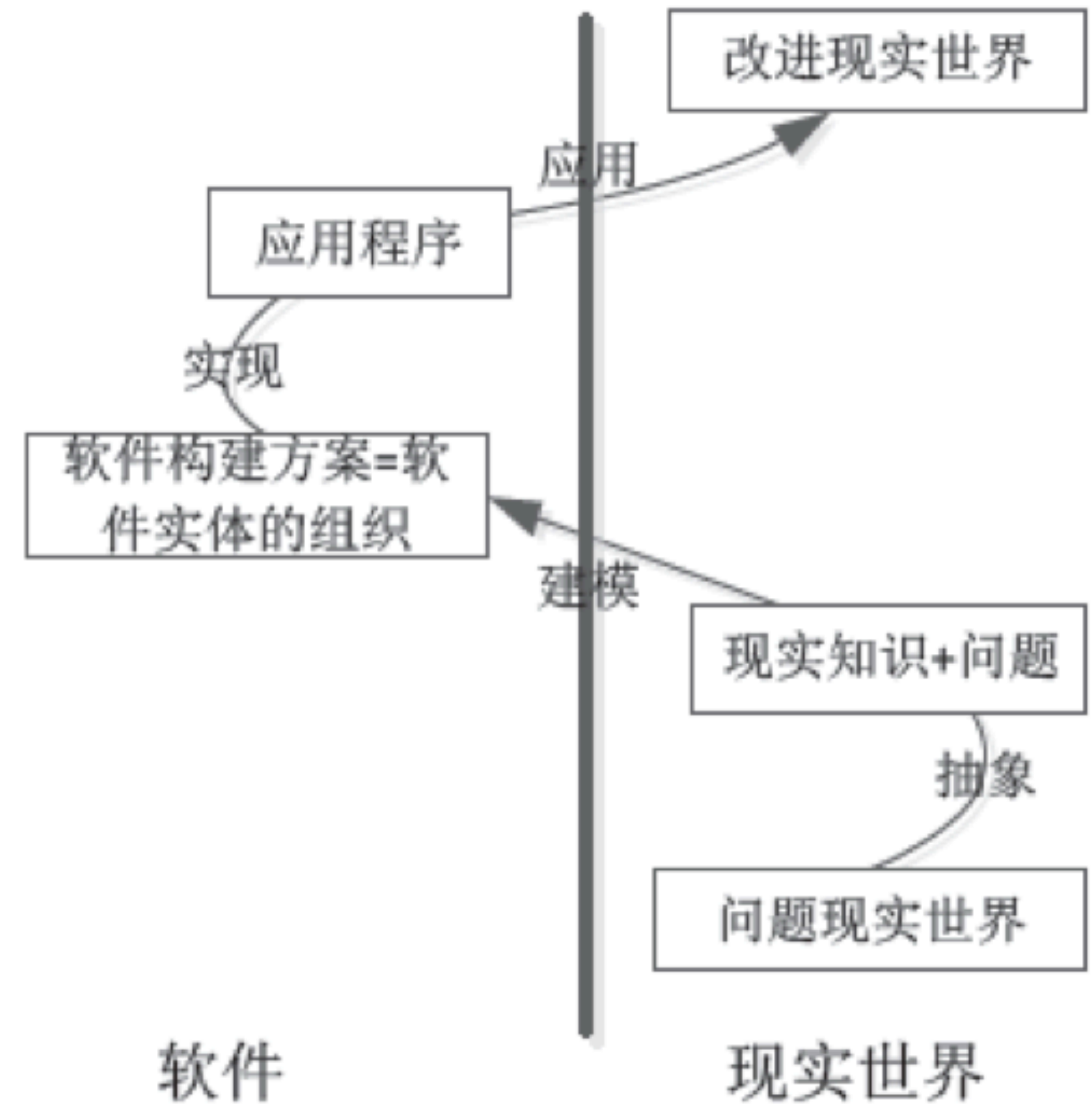
Types of software

- System software
 - 操作系统、数据库、数据仓库、嵌入式设备、安全
- Programming software
 - IDE、测试、持续集成、建模、度量
- Application software
 - 商业、政府、休闲、医疗、教育、国防、个人、专业、科学



应用软件基于现实又高于现实

- 应用软件被开发的目的是意图来源于现实世界的问题。
- 应用软件必须基于现实才能解决问题。
- 软件最终要被用于现实并改进现实。



What is software? - Summary

- Software is independent of hardware
- Software is a tool
- Software = programs + documents + data + knowledge
- Software development is much more complicated than programming
- Application software originate from the reality, and reversely improve the reality

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的~~概念~~
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

Software Engineering

What is software engineering?

Definition

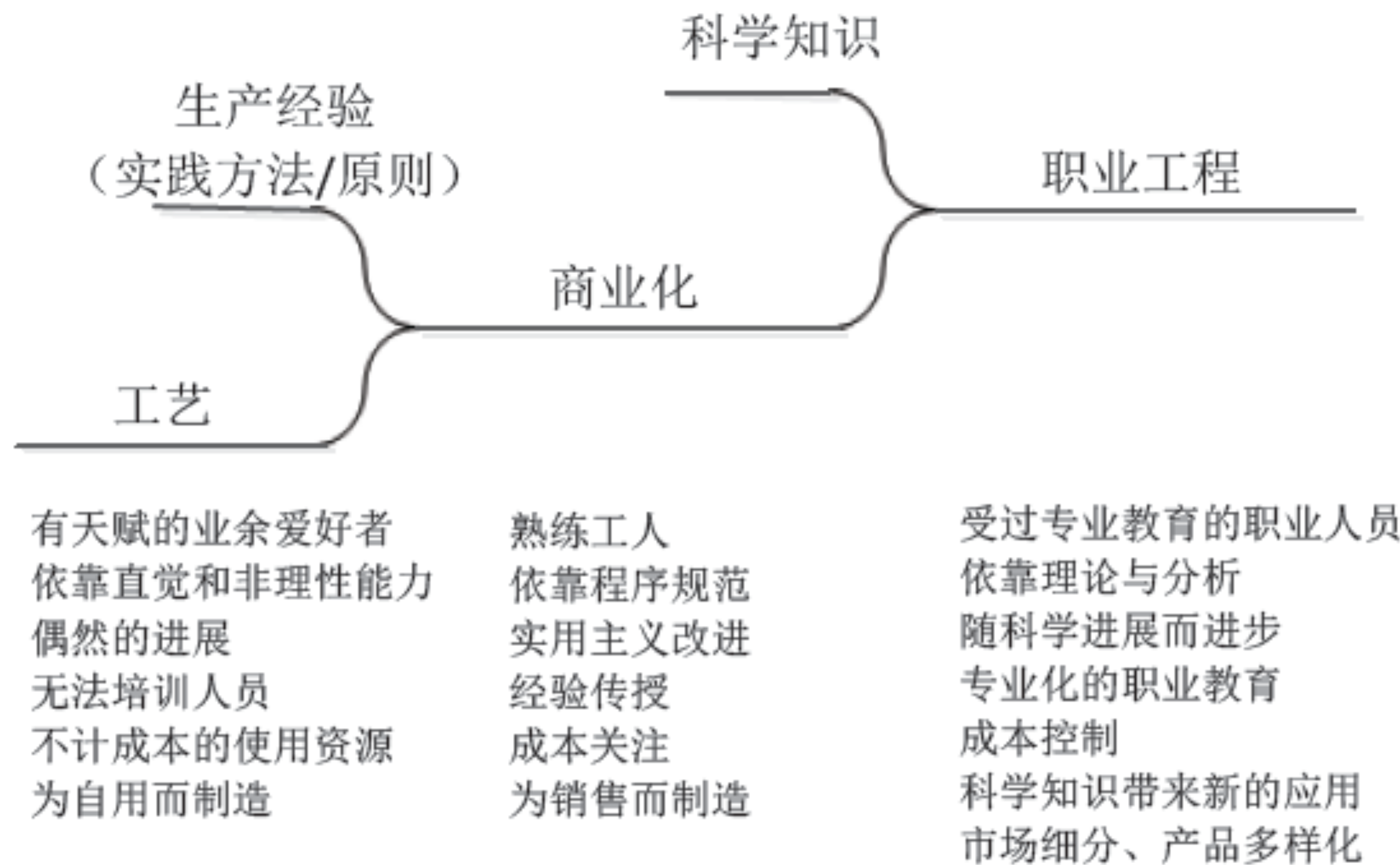
- [IEEE610.12-1990]:
 - (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.
 - (2) The study of approaches as in (1)

Engineering

- The American Engineers' Council for Professional Development (ECPD, the predecessor of ABET) has defined "engineering" as:
- The **creative application** of scientific principles to design or develop structures, machines, apparatus, or manufacturing processes, or works utilizing them singly or in combination; or to construct or operate the same with full cognizance of their design; or to forecast their behavior under specific operating conditions; all as respects **an intended function**, **economics** of operation or **safety** to life and property.

CCSE - About Engineer

- 工程师通过一系列的讨论决策，仔细评估项目的可选活动，并在每个决策点选择一种在当前环境中适合当前任务的方法进行工作。可以通过对成本和收益进行折衷分析调整相应策略。
- 工程师需要对某些对象进行度量，有时需要定量的工作；他们要校准和确认度量方法，并根据经验和实验数据进行估算。
- 软件工程师强调项目设计过程的纪律性，这是团队高效工作的条件。
- 工程师可胜任研究、开发、设计、生产、测试、构造、操作、管理，以及销售、咨询和培训等多种角色。
- 工程师们需要在某些过程中使用工具，选择和使用合适的工具是工程的关键要素。
- 工程师们通过专业协会发展和确认原理、标准和最佳实践方法，并提高个人能力。
- 工程师们能够重用设计和设计制品。



工程学科的发展

M. Shaw, Prospects for an Engineering Discipline of Software

- 具有解决实际问题的动机：
 - 工程学解决实际问题，而这些问题来源于工程领域之外的人——消费者。
- 应用科学知识指导工程活动：
 - 工程学不依赖于个人的技能，而是强调以科学知识为指导，按照特定方法与技术，进行规律性的设计、分析等活动，实现工程活动的可学习性和可重复性。
- 以成本效益比有效为基本条件：
 - 工程学不单单只是解决问题，它要有效利用所有资源，至少成本要低于效益，即成本效益比有效。
- 构建机器或事物：
 - 工程学强调构建实物工具，例如机器、事物等，并利用实物工具来解决问题。
- 以服务人类为最终目的：
 - 工程学考虑的不是单个客户的需要，而是要运用技术和经验实现全社会的进步。

Factor of Engineering

- Problem:
 - Motivation
- Scientific Knowledge:
 - Instrument
- Solution/Machine:
 - Objective
- Cost-Benefit Effective:
 - Condition

Software Engineering's Problem

- Real world chaos
 - If the state of the real world do not accord with our expectations , then there is a problem.
- Broad scope
 - All fields that mankind set foot in
 - In others engineering, problems are limited
- Mostly not clear, needed to be discovered
 - In others engineering, problems are specific and clear
- Engineering Idea
 - Objectives (Know what you want to)

Scientific Knowledge

- Computing science as the scientific basis
 - Formal techniques
 - Based on mathematics and logic
- Many aspects have been made systematic
 - Methods/methodologies/techniques
 - Languages
 - Tools
 - Processes

Engineering = Science + Principle + Art

- Science--Computing science knowledge is basis of SE
 - Features of the computer
 - Software entities (program languages)
 - Methods/methodologies/techniques
 - ...
- Principle--Practice knowledge should also be effectively shared
 - Methods/techniques
 - Problems/solutions
 - ...
- Art
 - Creativity: Analysis, Design

Solution/Machine && Product

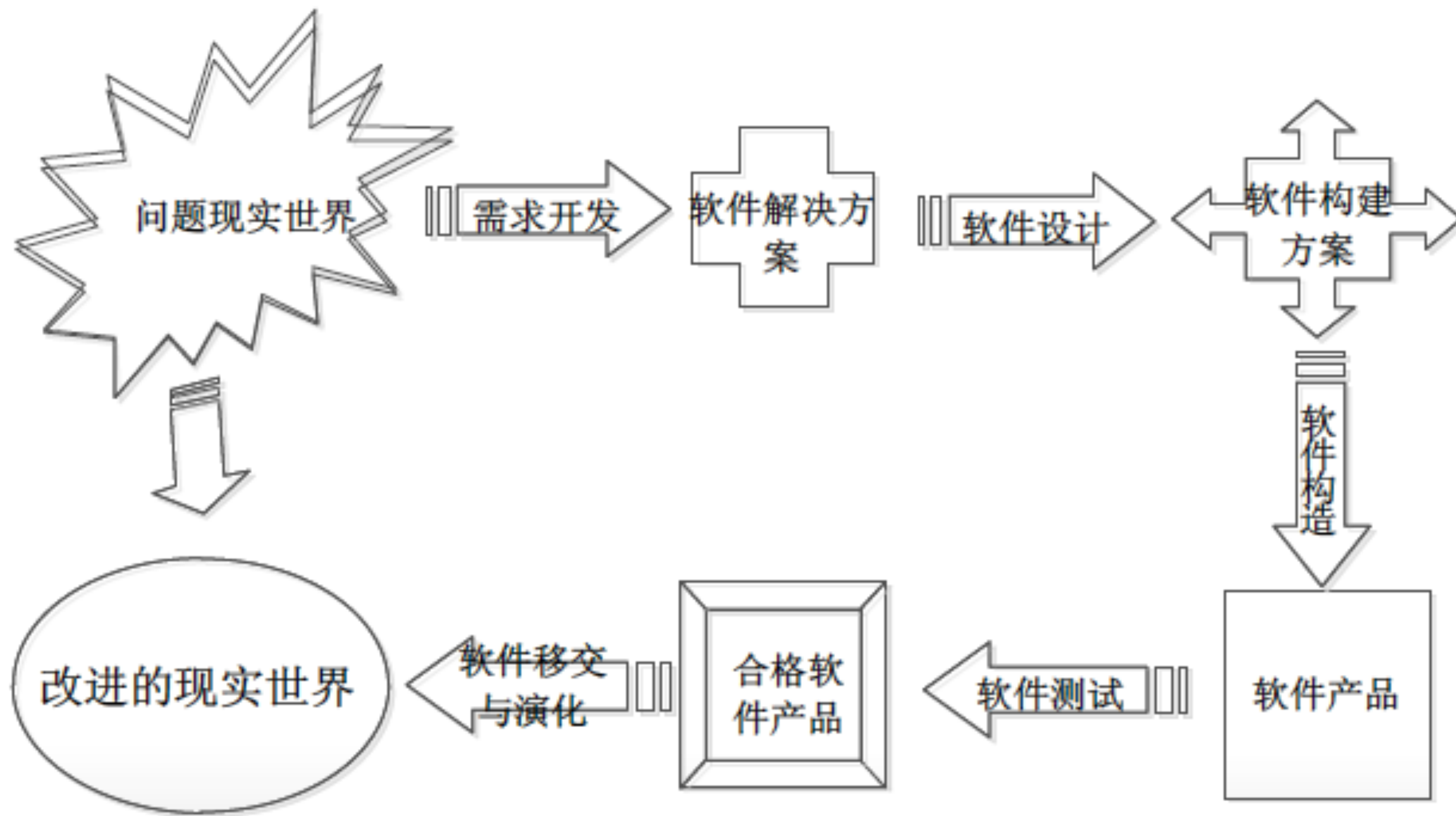
- Virtual Machine = General machine + Specific solution
- Solution can be mapped into the general machine with the help of program languages
- Not different machines for different problems, like others engineering
- Development of solution is the essential task of SE
- Mapping solution into general machine is accidental task of SE

Cost-Benefit Effective

- Feasibility study
 - Benefit is up to clients
 - Cost is up to software engineer
- There are always many ways to a destination, software engineers should choose the most cost-benefit effective one (not the most advanced one).
- When cost-benefit is inessential, then SE may disappears
 - Cost is small
 - Small program
 - Benefit is implicit
 - Exploration study
 - Cost is inessential
 - Computer science study
- Engineering Idea
 - Cost Benefit Effective
- Monitor and Control

Understanding of Software Engineering

- 软件工程是一种工程活动
- 软件工程的动机是解决实际问题
- 软件工程是科学性、实践性和工艺性并重的
- 软件工程追求足够好，不是最好
- 软件工程真正的产品是基于虚拟计算机的软件方案
- 软件工程的最终目的是要促进整个社会的进步



软件开发活动

Software Development Activity

	Target	Artifact
Software Requirement	What	SRS
Software Design	How	SDD
Software Construction	Build	Code and executable file
Software Testing	Are you building the “right” thing? Are you building it “right”?	Test Report
Software Deliver	Install	User Document and System Document
Software Maintenance	Revolution	New version software

角色分工

- 需求工程师，又被称为需求分析师：
 - 承担需求开发任务。软件产品的需求开发工作通常由多个需求工程师来完成，他们共同组成一个需求工程师小组，在首席需求工程师的领导下开展工作。通常一个团队只有一个需求工程师小组。
- 软件体系结构师：
 - 承担软件体系结构设计任务。通常也是由多人组成一个小组，并在首席软件体系结构师的领导下开展工作。通常一个团队只有一个软件体系结构师小组。
- 软件设计师：
 - 承担详细设计任务。在软件体系结构设计完成之后，可以将其部件分配给不同的开发小组。开发小组中负责所分配部件详细设计工作的人员就是软件设计师。一个团队可能有一个或多个开发小组。一个小组可能有一个或多个软件设计师。

角色分工

- 程序员：
 - 承担软件构造任务。程序员与软件设计师通常是同一批人，也是根据其所分配到的任务开展工作。
- 人机交互设计师：
 - 承担人机交互设计任务。人机交互设计师与软件设计师可以是同一批人，也可以是不同人员。在有多个小组的软件工程团队中，可以有一个单独的人机交互设计师小组，也可以将人机交互设计师分配到各个小组。
- 软件测试人员：
 - 承担软件测试任务。软件测试人员通常需要独立于其他的开发人员角色。一个团队可能有一个或多个测试小组。一个小组可能有一个或多个软件测试人员。

角色分工

- 项目管理人员：
 - 负责计划、组织、领导、协调和控制软件开发的各项工作。相比于传统意义上的管理者，他们不完全是监控者和控制者，更多得是协调者。通常一个团队只有一个项目管理人员。
- 软件配置管理人员：
 - 管理软件开发中产生的各种制品，具体工作是对重要制品进行标识、变更控制、状态报告等。通常一个团队只有一个软件配置管理人员。
- 质量保障人员：
 - 在生产过程中监督和控制软件产品质量的人员。通常一个团队有一个质量保障小组，由一个或多个人员组成。
- 培训和支持人员：
 - 负责软件移交与维护任务。他们可以是其他开发人员的一部分，也可以是独立的人员。
- 文档编写人员：
 - 专门负责写作软件开发各种文档的人员。他们的存在是为了充分利用部分宝贵的人力资源（例如需求工程师和软件体系结构师），让这些人力资源从繁杂的文档化工作中解放出来。

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

Knowledge Area

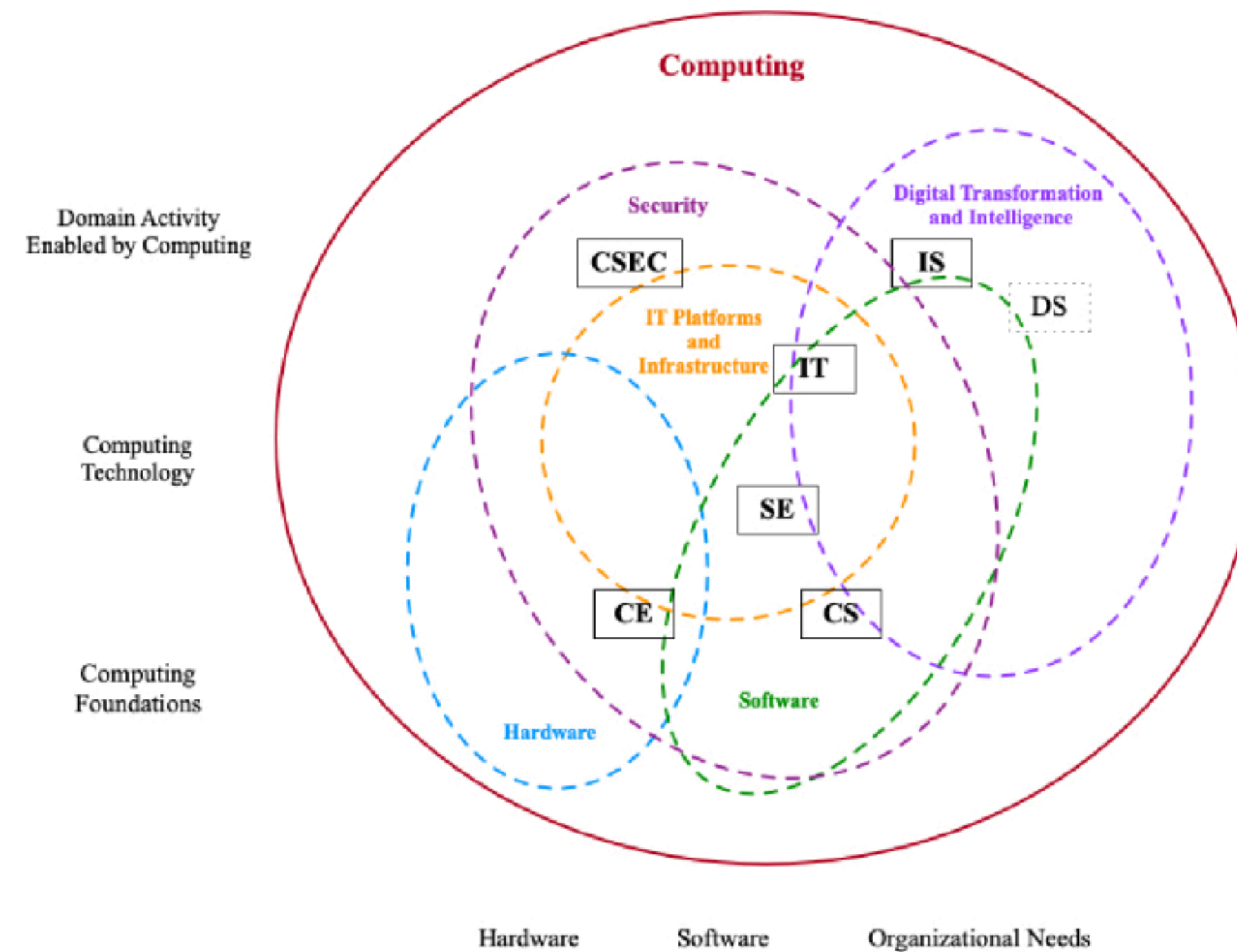
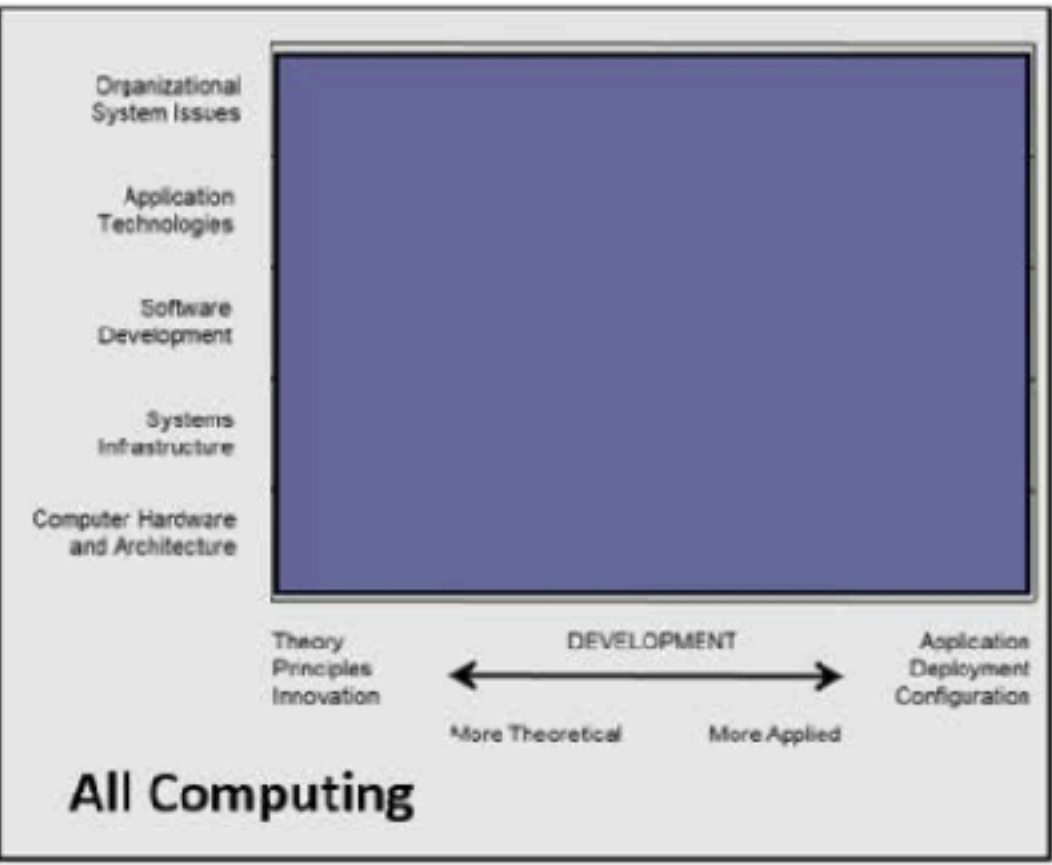
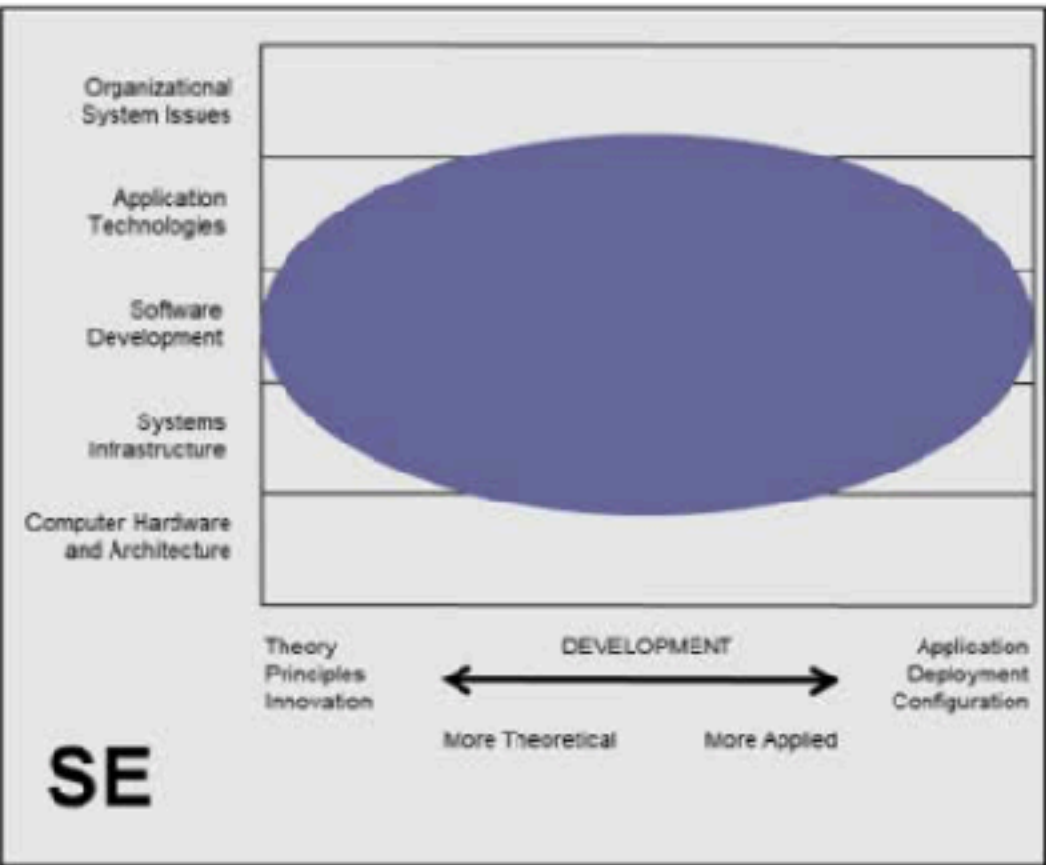
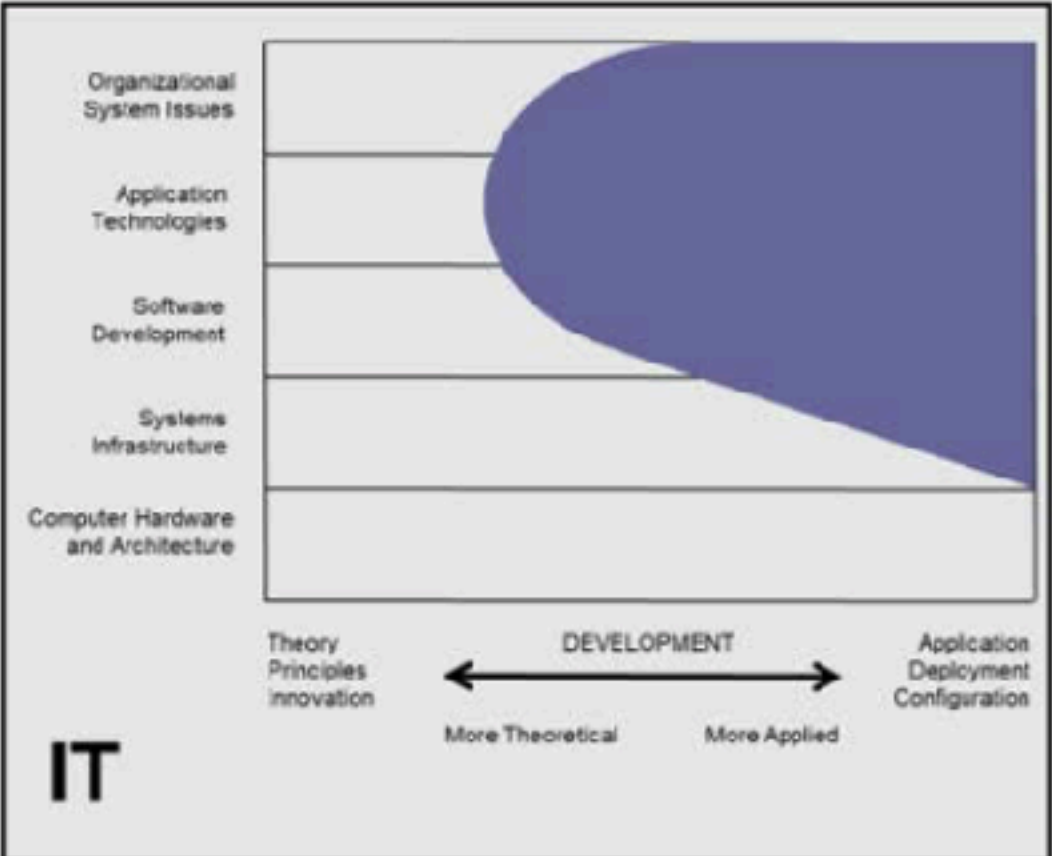
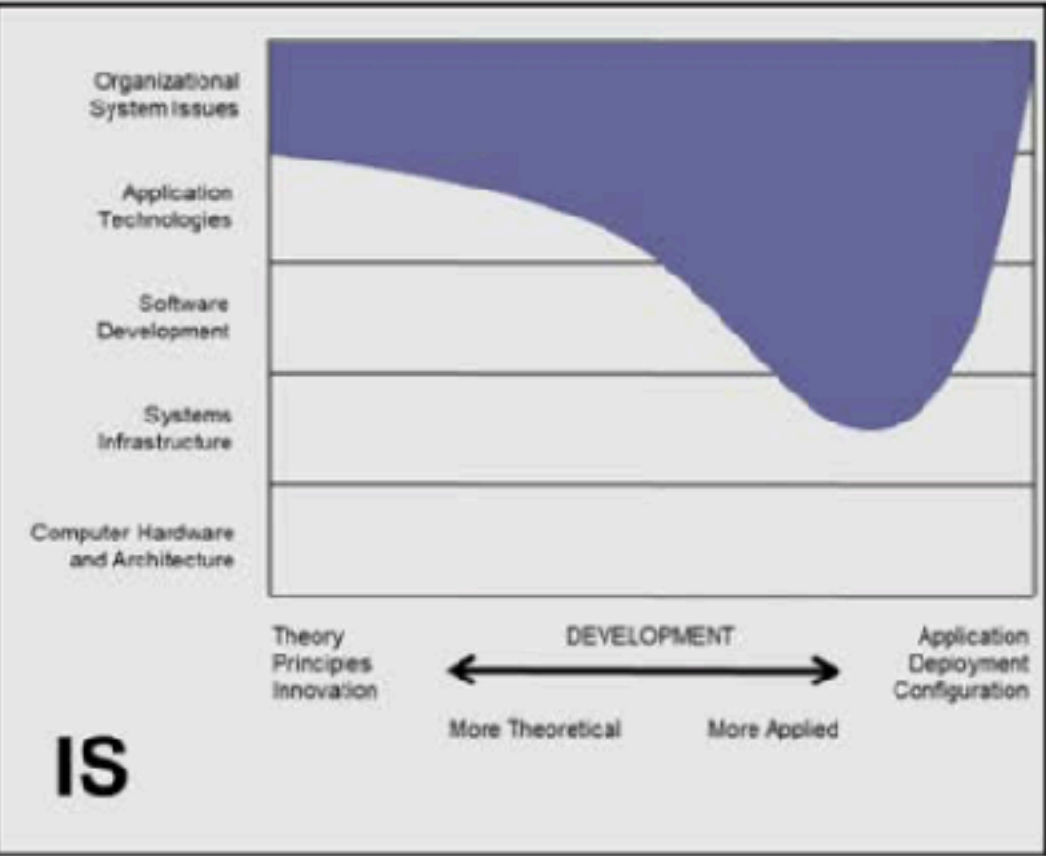
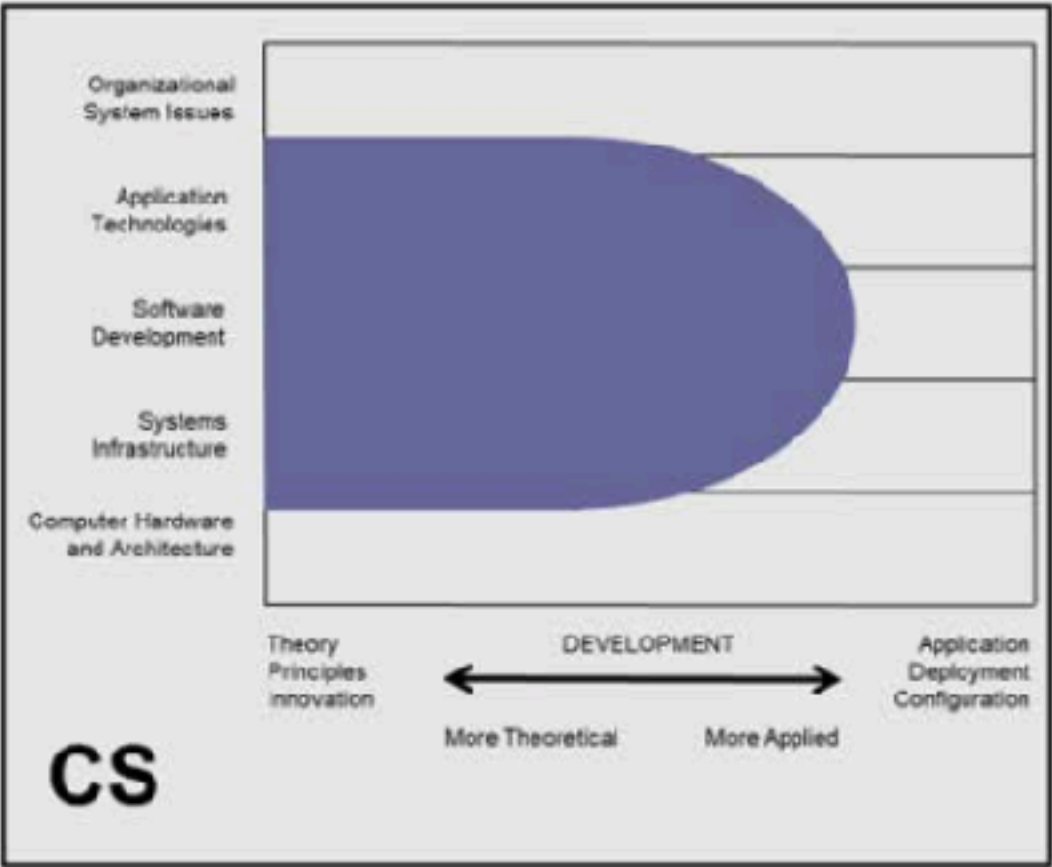
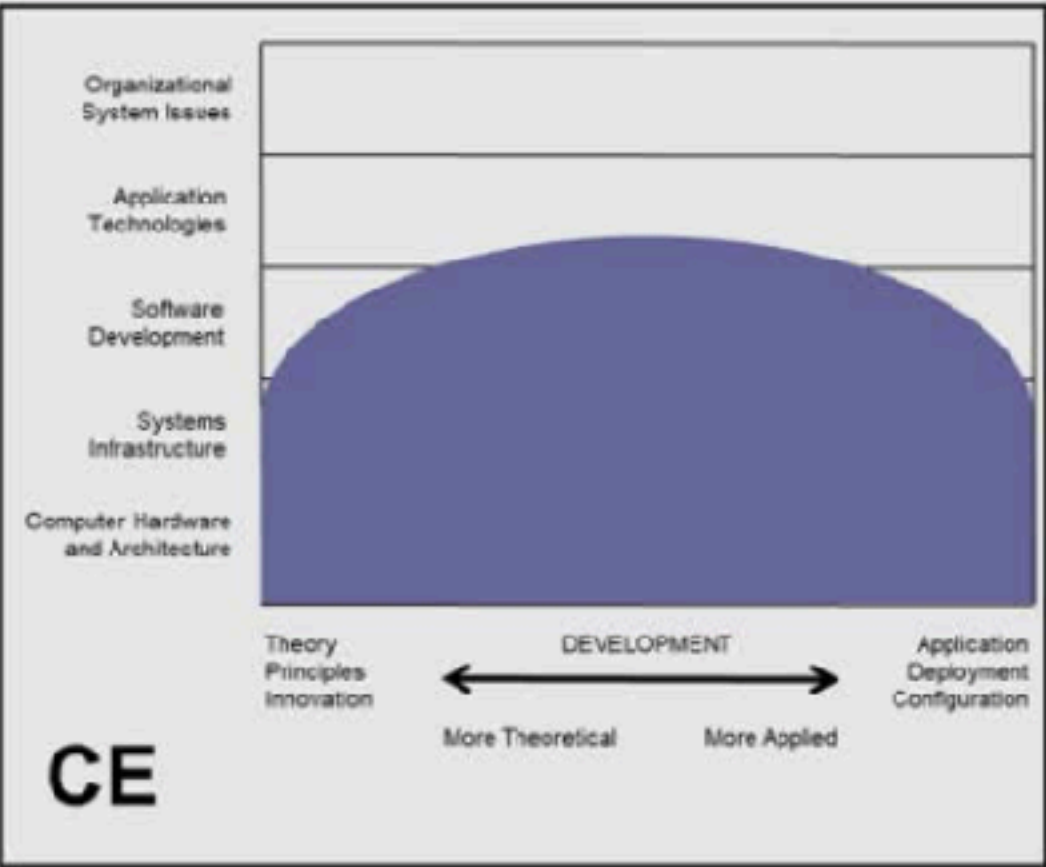


Figure 2.2. A contemporary view of the landscape of computing education

Legend: Curricular reports: CE=computer engineering; CS=computer science; CSEC=cybersecurity; IS=information systems; IT=information technology; SE=software engineering; DS=data science (under development)

Landscape of Computing education



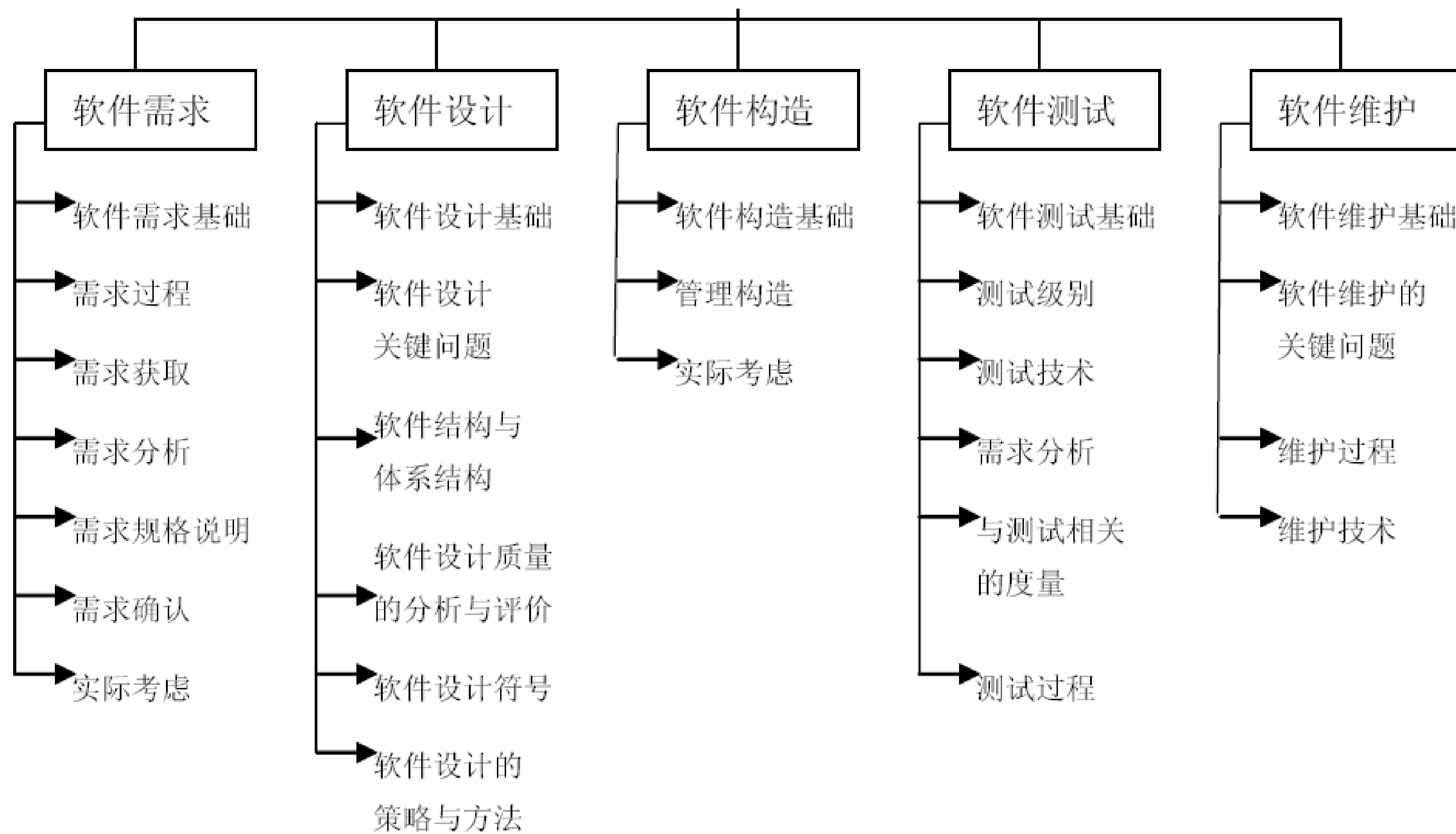
Landscape of Computing Knowledge

Table 5.3 Landscape of Computing Knowledge

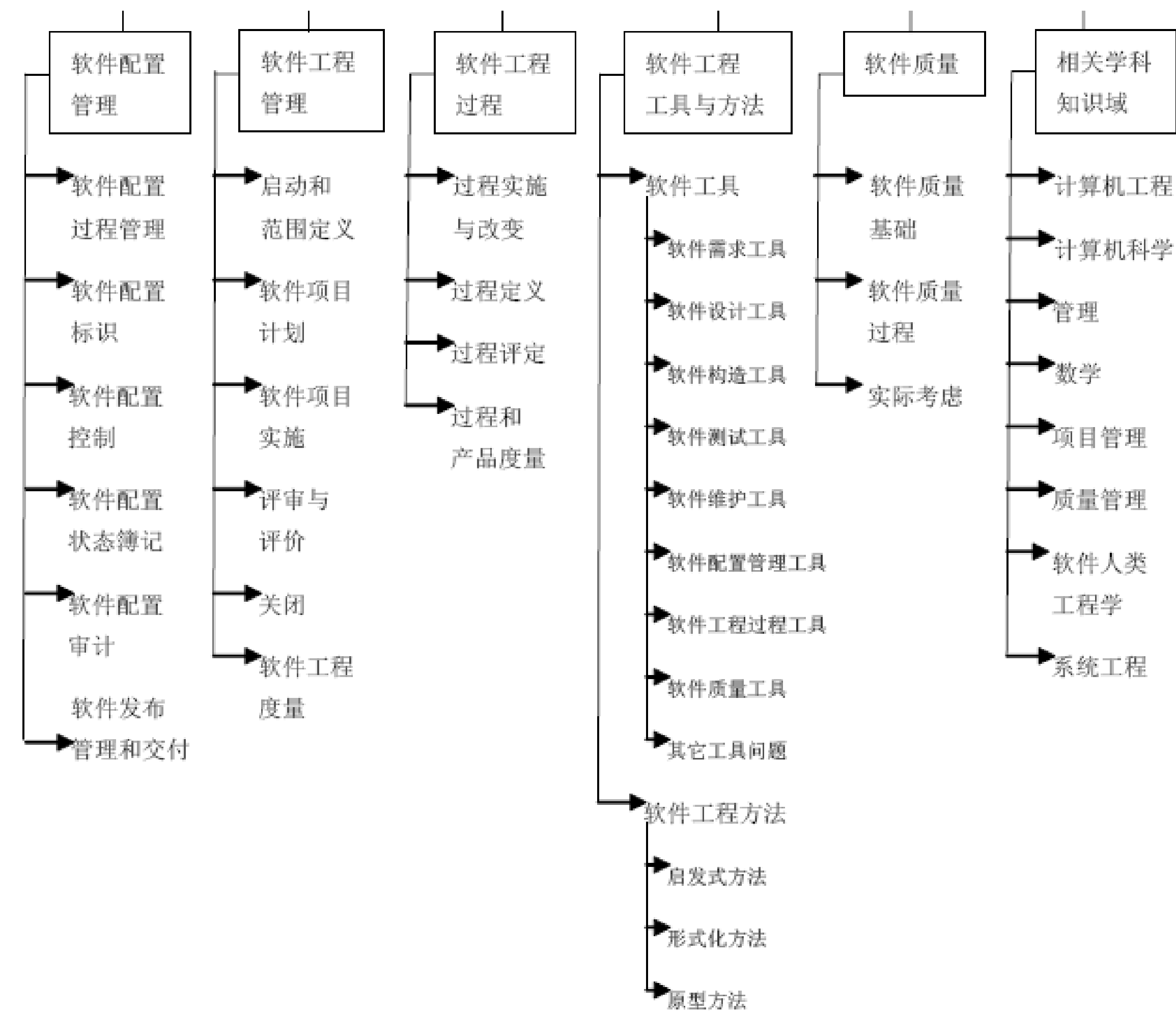
		CE		CS		CSEC		IS		IT		SE	
		Min	Max	Min	Max	Min	Max	Min	Max	Min	Max	Min	Max
1. Users and Organizations	1.1. Social Issues and Professional Practice	2	5	2	4	2	4	3	5	2	4	3	5
	1.2. Security Policy and Management	1	3	2	3	4	5	2	3	2	4	2	4
	1.3. IS Management and Leadership	0	2	0	2	1	2	4	5	1	2	1	2
	1.4. Enterprise Architecture	0	1	0	1	1	2	3	5	1	3	1	3
	1.5. Project Management	1	3	2	3	1	2	4	5	2	3	2	4
	1.6. User Experience Design	1	3	2	4	1	3	2	4	3	4	3	5
2. Systems Modeling	2.1. Security Issues and Principles	2	3	2	3	4	5	2	4	3	4	2	4
	2.2. Systems Analysis & Design	1	2	1	2	1	2	4	5	1	3	2	4
	2.3. Requirements Analysis and Specification	1	2	1	2	0	2	2	4	1	3	3	5
	2.4. Data and Information Management	1	2	2	4	2	3	3	5	2	3	2	4
3. Systems Architecture and Infrastructure	3.1. Virtual Systems and Services	1	3	1	3	1	2	1	2	3	4	1	3
	3.2. Intelligent Systems (AI)	1	3	3	5	1	2	1	2	1	2	0	1
	3.3. Internet of Things	2	4	0	2	1	3	1	3	2	4	1	3
	3.4. Parallel and Distributed Computing	2	4	2	4	1	2	1	3	1	3	2	3
	3.5. Computer Networks	2	4	2	4	2	4	1	3	3	4	2	2
	3.6. Embedded Systems	3	5	0	2	1	3	0	1	0	1	0	3
	3.7. Integrated Systems Technology	1	2	0	2	0	2	1	3	3	4	1	3
	3.8. Platform Technologies	0	1	1	2	1	2	1	3	2	4	0	2
	3.9. Security Technology and Implementation	2	3	2	4	4	5	1	3	2	4	2	4
4. Software Development	4.1. Software Quality, Verification and Validation	1	3	1	3	1	2	1	3	1	2	3	5
	4.2. Software Process	1	2	1	3	0	2	1	3	1	3	3	5
	4.3. Software Modeling and Analysis	1	3	1	3	1	2	2	4	1	3	4	5
	4.4. Software Design	2	4	2	4	1	3	1	3	1	2	4	5
	4.5. Platform-Based Development	0	2	2	4	0	1	1	3	2	4	1	3
5. Software Fundamentals	5.1. Graphics and Visualization	1	2	2	4	0	1	1	1	0	1	0	2
	5.2. Operating Systems	2	4	3	5	2	3	1	2	1	3	1	3
	5.3. Data Structures, Algorithms and Complexity	2	4	4	5	1	3	1	3	1	2	2	4
	5.4. Programming Languages	2	3	3	5	1	2	1	2	1	2	2	3
	5.5. Programming Fundamentals	2	4	4	5	2	3	1	3	2	4	3	5
	5.6. Computing Systems Fundamentals	2	3	2	3	1	2	2	3	1	3	2	3
6. Hardware	6.1. Architecture and Organization	4	5	3	4	1	3	1	2	1	2	1	3
	6.2. Digital Design	4	5	1	2	0	2	0	1	0	1	0	2
	6.3. Circuits and Electronics	4	5	1	2	0	1	0	1	1	2	0	1
	6.4. Signal Processing	3	4	0	1	0	2	0	1	0	1	0	1

Knowledge Area

- SWEBOK
 - The Software Engineering Body of Knowledge (SWEBOK) is a product of the Software Engineering Coordinating Committee sponsored by the IEEE Computer Society



SWEBOK V3 软件工程技术知识域



SWEBOK V3软件工程管理知识域

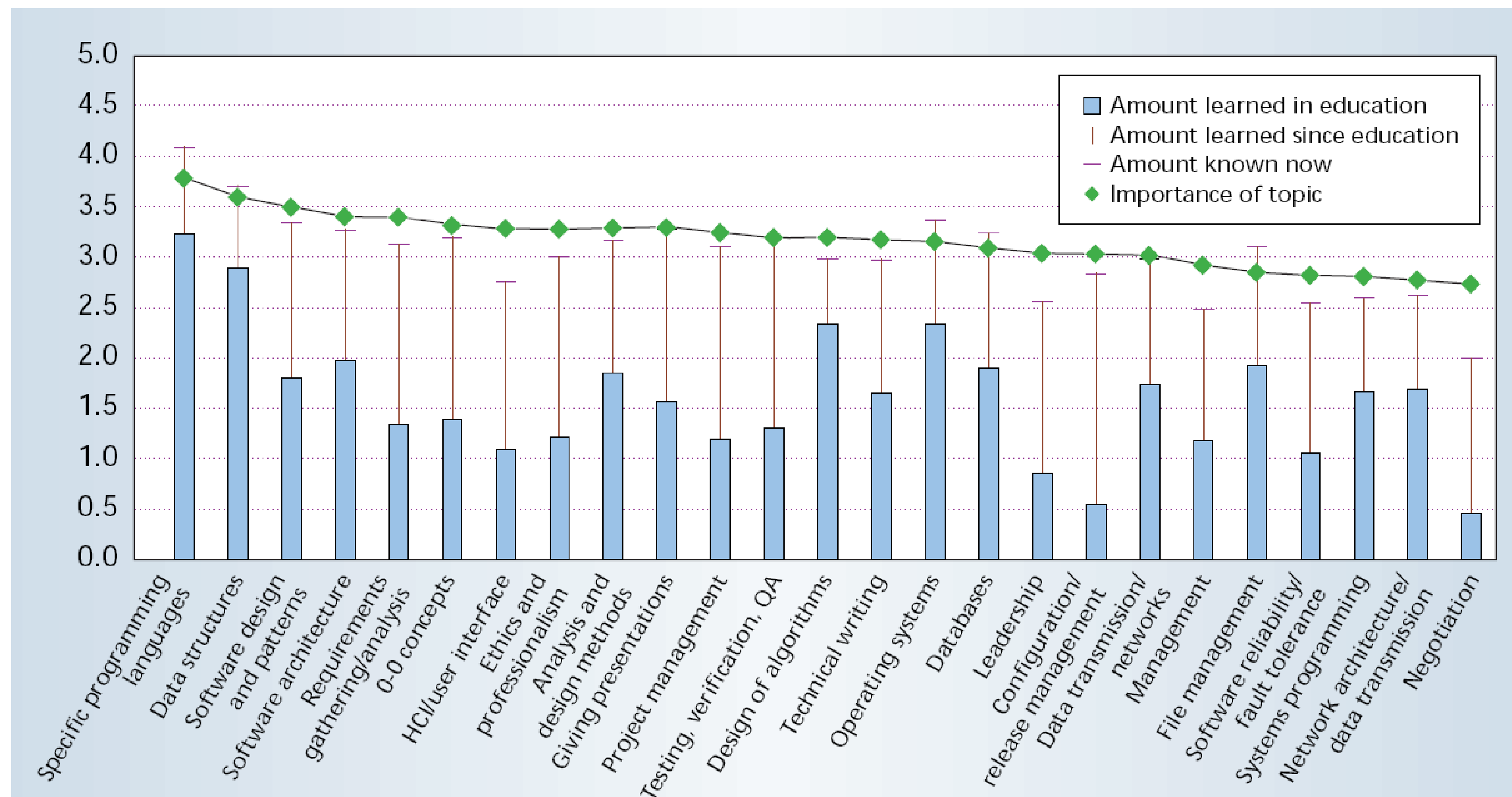
IEEE-CS Professional & Educational Activities Board (PEAB) SWEBOK Evolution Team
(Vice President for PEAB and Team Chair: Hironori Washizaki)

SWEBOK V3	SWEBOK V4	Public review
Introduction	Introduction	Open
1. Software Requirements	1. Software Requirements	closed
	2. Software Architecture	closed
2. Software Design	3. Software Design	closed
3. Software Construction	4. Software Construction	closed
4. Software Testing	5. Software Testing	closed
	6. Software Engineering Operations	Open
5. Software Maintenance	7. Software Maintenance	Open
6. Software Configuration Management	8. Software Configuration Management	Open
7. Software Engineering Management	9. Software Engineering Management	closed
8. Software Engineering Process	10. Software Engineering Process	Open
9. Software Engineering Models and Methods	11. Software Engineering Models and Methods	Open
10. Software Quality	12. Software Quality	closed
	13. Software Security	closed
11. Software Engineering Professional Practice	14. Software Engineering Professional Practice	closed
12. Software Engineering Economics	15. Software Engineering Economics	closed
13. Computing Foundations	16. Computing Foundations	Open
14. Mathematical Foundations	17. Mathematical Foundations	
15. Engineering Foundations	18. Engineering Foundations	
Appendix A. Knowledge Area Specifications	Appendix A. Knowledge Area Specifications	Open
Appendix B. Standards	Appendix B. Standards	

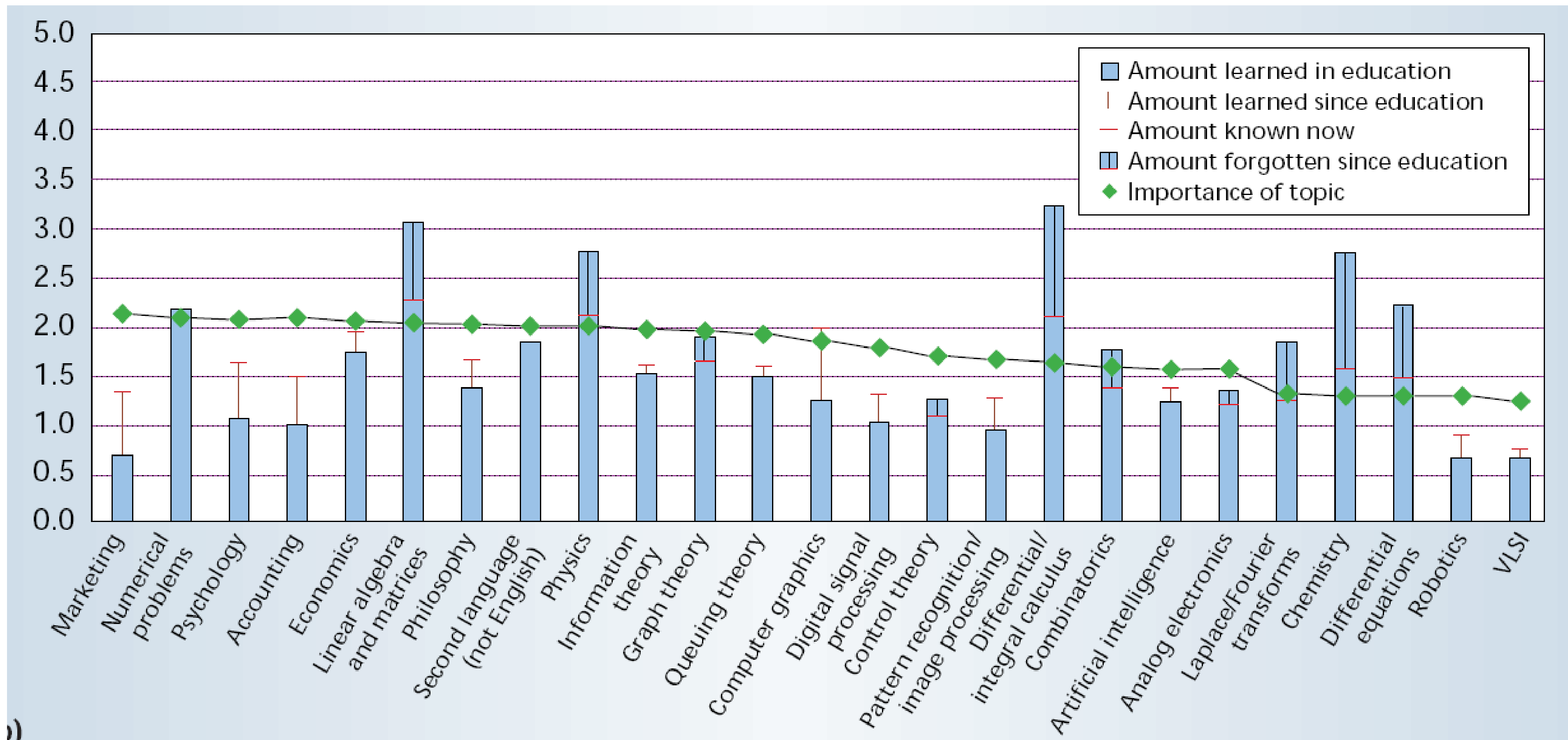
额外增加了:

AI for SE

SE for AI



职业软件工程师的软件工程知识重要性评价 I



职业软件工程师的软件工程知识重要性评价 II

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

项目和项目管理

项目

- 项目是具有下列特征的一系列活动和任务[Kerzner2009]:
 - 具有一个明确的目标;
 - 有限定的开始和结束日期;
 - 有成本限制;
 - 消耗人力和非人力资源;
 - 多工种合作。

项目管理的目标

- 项目管理的目标是做到以下方面：
 - 在限定时间内；
 - 在一定的成本内；
 - 在要求的质量水平上；
 - 高效使用资源；
 - 获得客户认可。

过程组与活动

- 过程组：
 - 项目启动、项目计划、项目执行、项目跟踪与控制 and 项目收尾
- 活动
 - 计划制定、团队管理、成本控制、质量保障、度量、过程管理、进度跟踪与控制、风险管理、配置管理

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

团队

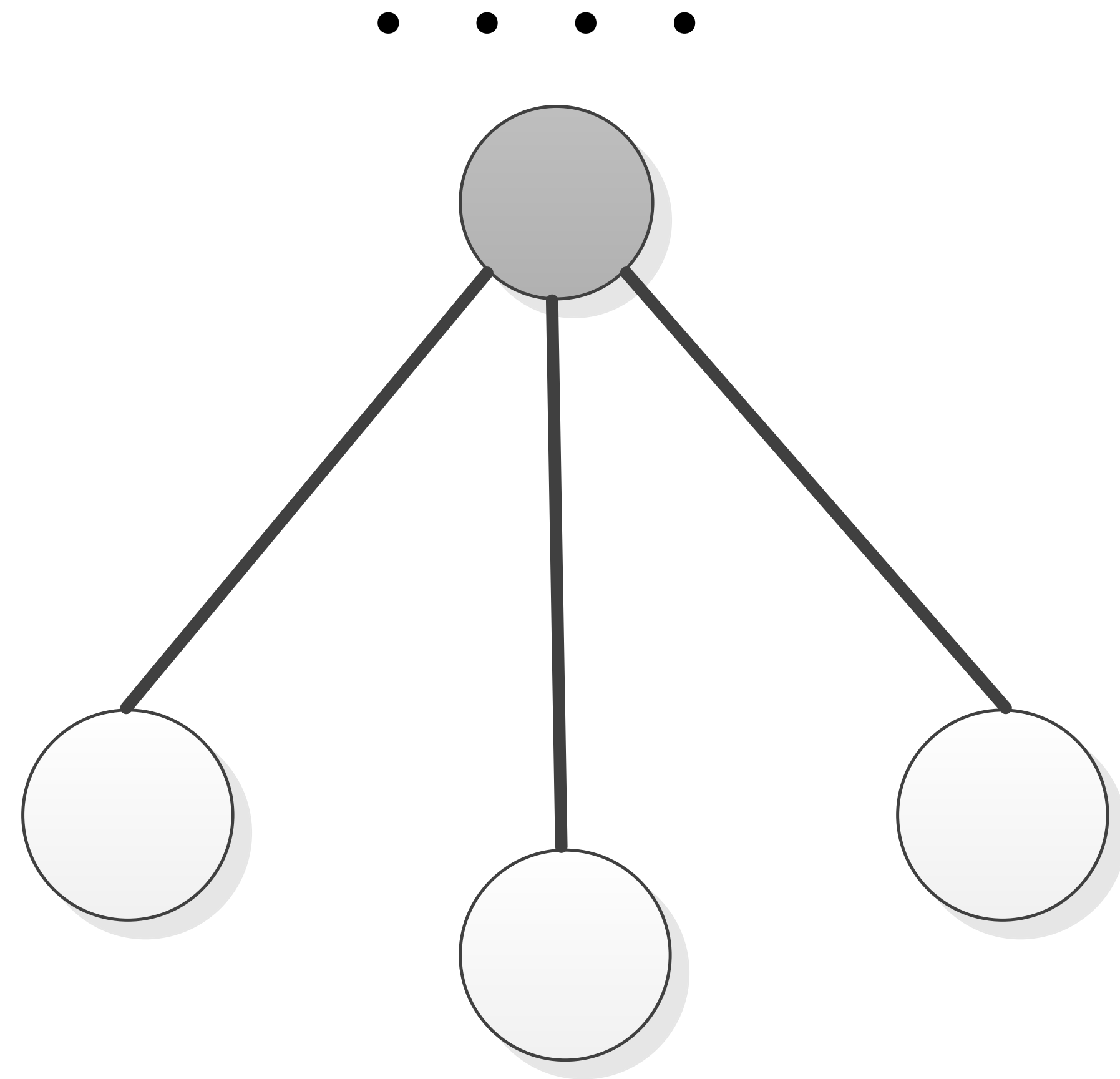
- 一个协作良好的团队是任何项目成功的基础。
- 软件项目尤其依赖于有效的团队组织和管理：软件开发是一个以人为主的活动，人力资源是软件项目最大的资产。
- 有很多实践者认为比生产高质量产品更大的成功是在生产过程中建立一个凝聚的团队

团队的特征

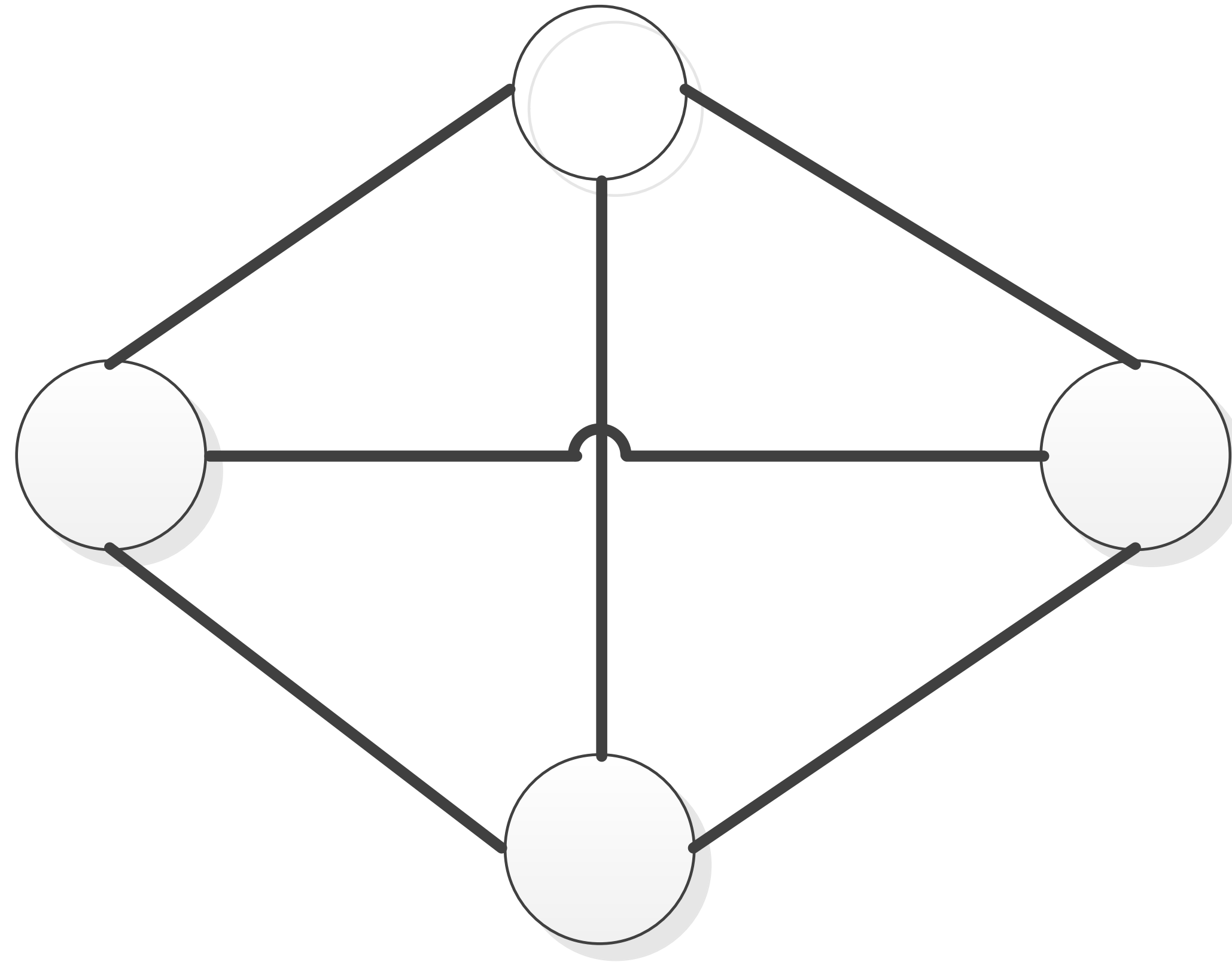
- [Katzenbach1993]将团队定义为：为了一致的目的、绩效标准、方法而共担责任并且技能互补的少数人。
- 团队成员要具备共同的目标。
- 团队成员要共担责任。
- 团队成员要技能互补。
- 团队内部要有一个明确的结构。

团队结构

- 主程序员团队
- 民主团队
- 开放团队



主程序员团队



民主团队



开放团队

团队建设

- 建立团队章程
- 持续成功
- 和谐沟通
- 避免团队杀手

某小组的团队章程示例

- 开会规则：

- 1、开会时间由组员协商决定（电话协商），开会地点为509，一旦决定不允许请假、缺席。否则要求请其他小组成员喝冷饮，并组内批评。
- 2、会前各小组成员要对所要讨论的议题有所准备，各小组成员均需发表个人看法，会议结果由表决决定，组长两票，组员各一票。会议期间需要有完整的记录。
- 3、开会期间务须认真，不允许干任何于会议内容无关的事情。否则会内批评。

- 组内联系方式：

- 开会协商：电话、当面协商； 会议缺席、迟到：电话联系； 文档交流：邮箱、svn； 娱乐交流：qq； 项目产物交流：svn；

- 工作时间安排：

- 上午： 9：00——11:30 ； 下午： 14:00——17:00 ； 晚上： 19:00——21:00.
- 周六、周日的上午跟晚上休息。 工作日的晚上如果项目进度较快可以选择性休息，但一周不允许超过两次。
- 如有无故旷课或早退现象需要写检讨并接受小组批评；如果早上迟到需要为大家带冷饮，迟到不允许超过40分钟，否则视为旷课。

顺序	开发人员	项目管理人员	普通人
1	成就感	责任感	成就感
2	发展机遇	成就感	受认可程度
3	工作乐趣	工作乐趣	工作乐趣
4	个人生活	受认可程度	责任感
5	成为技术主管的机会	发展机遇	领先
6	领先	与下属关系	工资
7	同事间人际关系	同事间人际关系	发展机遇
8	受认可程度	领先	与下属关系
9	工资	工资	地位
10	责任感	操控能力	操控能力
11	操控能力	公司政策和经营	同事间人际关系
12	工作保障	工作保障	成为技术主管的机会
13	与下属关系	成为技术主管的机会	公司政策和经营
14	公司政策和经营	地位	工作条件
15	工作条件	个人生活	个人生活
16	地位	工作条件	工作保障

不同人员激励因素比较， 源自[Boehm1981]

避免团队杀手

- [DeMarco1999]认为组织和管理团队时要回避下列团队杀手：
 - 防范式管理。
 - 官僚主义。
 - 地理分散。
 - 时间分割。
 - 产品质量的降低。
 - 虚假的最后期限。
 - 小圈子控制。

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

软件质量

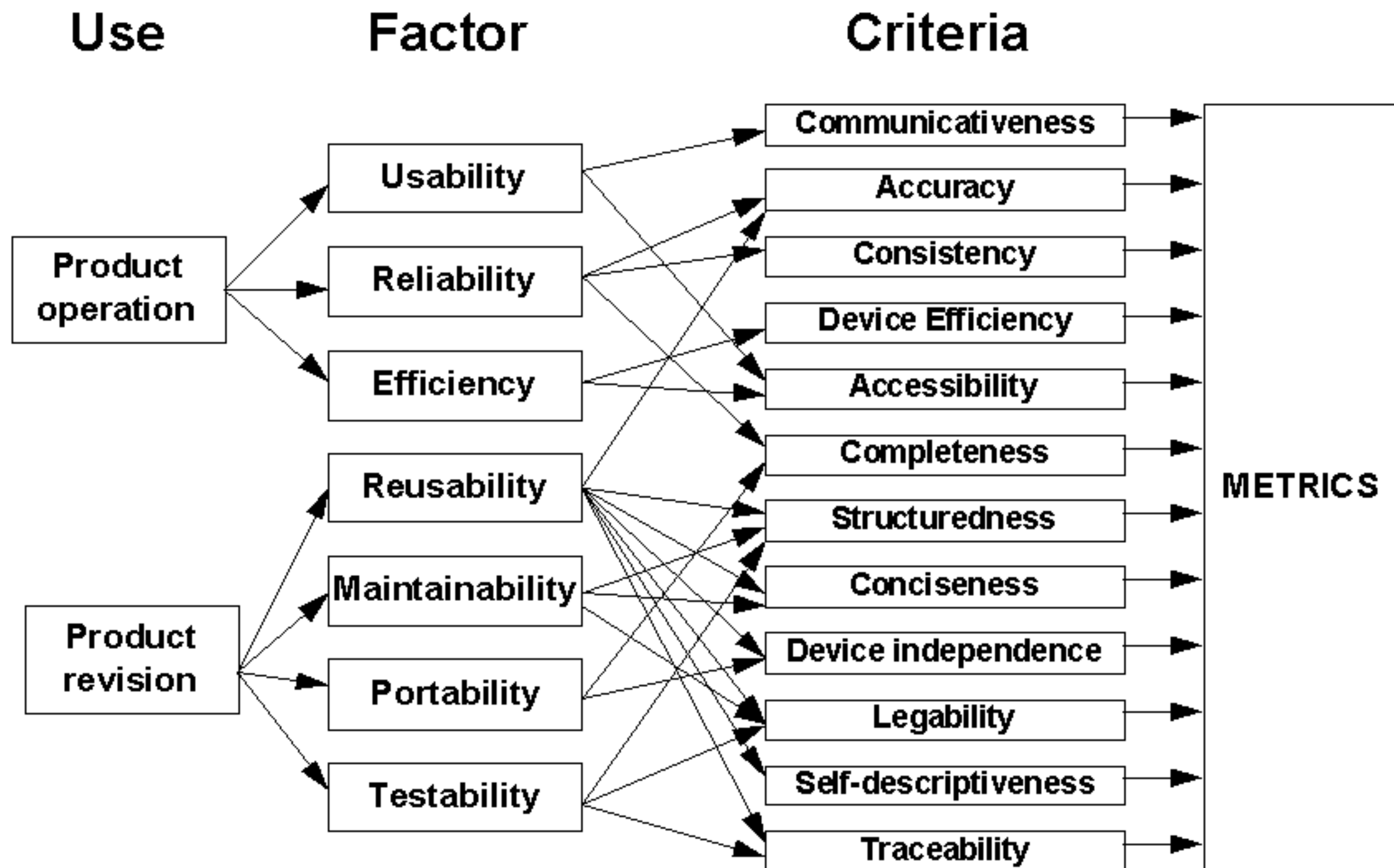
- 软件工程师也要对软件产品的质量负责。
- 对软件质量的要求可能是显式的，也可能是隐式的。
- 人们通常会选用系统的某些质量要素进行量化处理，建立质量特征，这些特征被称为质量属性（Quality Attribute）。
- 为了根据质量属性描述和评价系统的整体质量，人们从很多质量属性的定义当中选择了一些能够相互配合、相互联系的特征集，它们被称为质量模型。

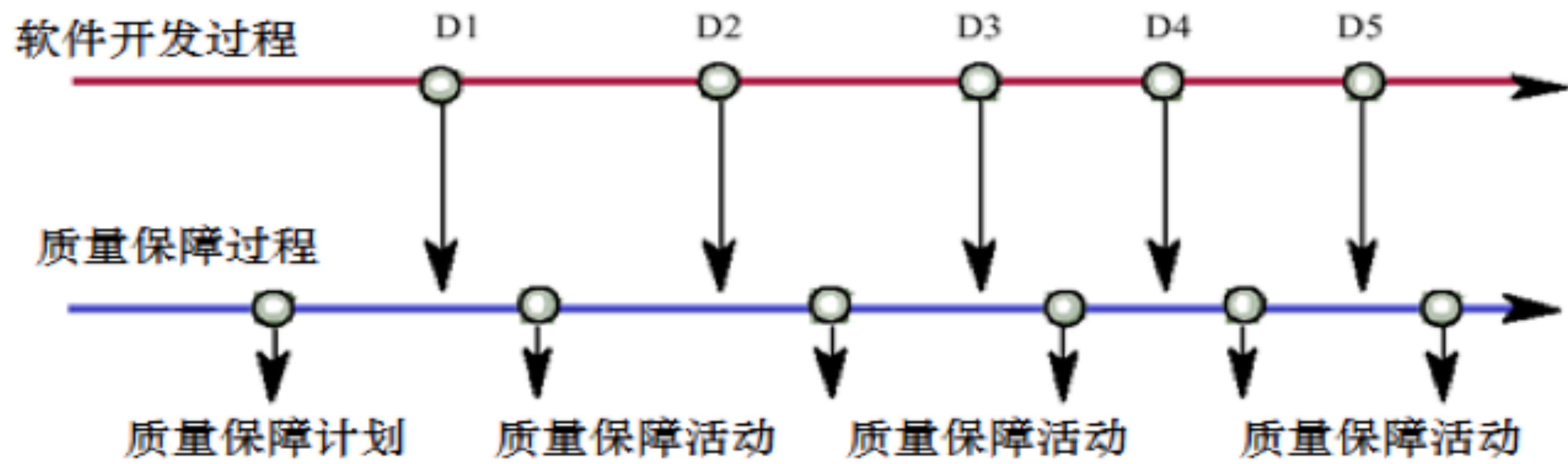
质量模型

- [IEEE1061-1992,1998] 和 [ISO/IEC 9126-1]

- 因素

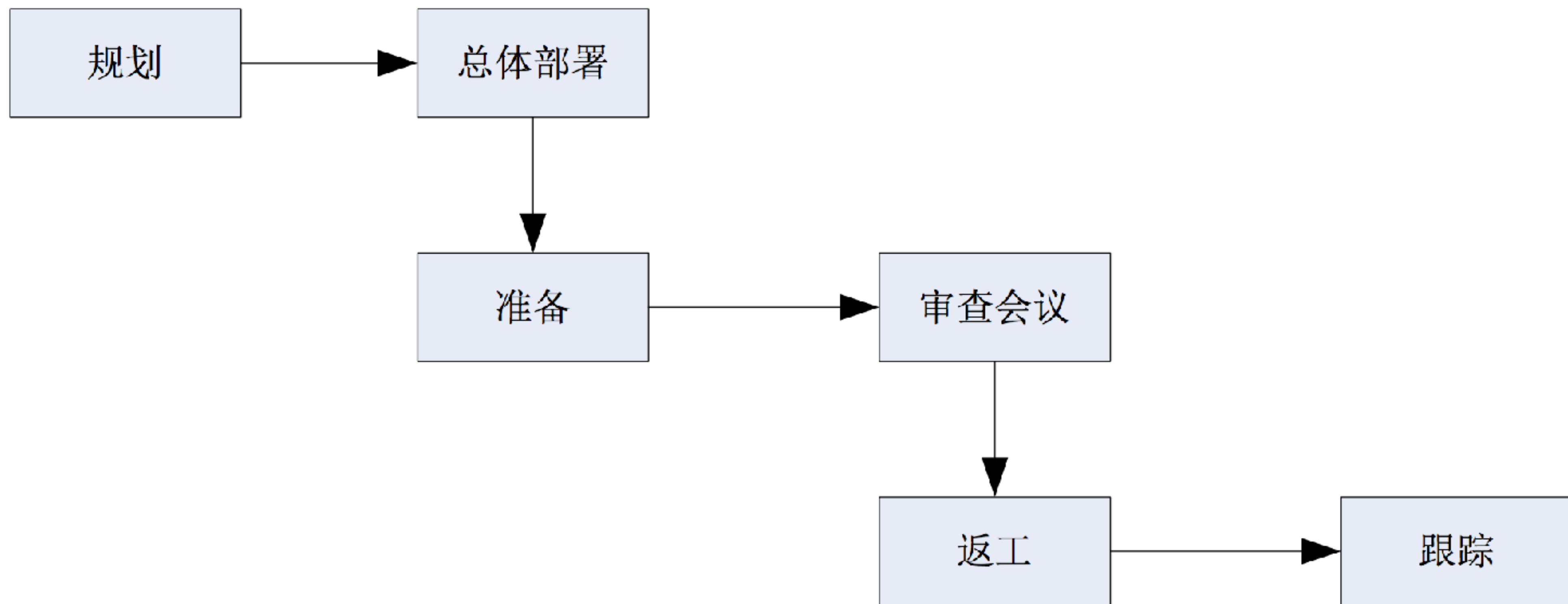
- 功能性
- 可靠性
- 易用性
- 效率
- 可维护性
- 可移植性





质量保障

里程碑	质量保障活动
需求开发	需求评审、需求度量
体系结构	体系结构评审、集成测试（持续集成）
详细设计	详细设计评审、设计度量、集成测试（持续集成）
实现	代码评审、代码度量、测试（测试驱动、持续集成）
测试	测试、测试度量



评审过程

评审

- 1. 在规划阶段（Planning），制定审查计划，决定审查会议的次数，安排每次审查会议的时间、地点、参与人员、审查内容等等。
- 2. 在总体部署阶段（Overview），向所有参与审查会议的人员描述待审查材料的内容、审查的目标以及一些假设，并分发文档。
- 3. 在准备阶段（Preparation），审查人员各自独立执行检查任务。在检查的过程当中，他们可能会被要求使用检查清单、场景等检查方法。检查中发现的问题会被记录下来，以准备开会讨论或者提交给收集人员。
- 4. 在审查会议阶段（Inspection Meeting），通过会议讨论，识别、确认、分类发现的错误。
- 5. 在返工阶段（Rework），修改发现的缺陷。
- 6. 在跟踪阶段（Follow-up），要确认所有发现的问题都得到了解决，所有的错误都得到了修正。

质量度量

- 度量产生自统计控制（Statistical Control）思想。“你不能控制自己无法度量的东西”[DeMarco1998]。
- 测度（Measure）就是为了描述软件产品而提供的定量指标。
 - 代码行数
- 进行测度的活动被称为测量（Measurement）。
- 度量（Metric）是软件产品在特定属性上的量化测度程度。
 - 例如基于所有对象的代码行数测度可以建立平均代码行数、最大代码行数、最小代码行数

Outline

- 软件工程基础
 - 软件的概念
 - 软件工程的观念
 - 软件工程知识域
- 项目管理基础
 - 项目和项目管理
 - 团队组织与管理
 - 软件质量保障
 - 软件配置管理

软件配置管理的动机

- 在软件开发活动中，除了最终产品之外，还会产生很多中间制品，例如需求规格说明、需求分析模型、软件体系结构设计模型、详细设计模型等。这些制品是不同阶段、不同角色、不同软件开发活动进行协同的基础。
- 在复杂软件系统开发中，产生的制品数量众多，以至于开发者需要维护一个清单才能清楚项目所处的状态，理解已经完成的工作和将要进行的工作。
- 某个制品发生变化带来的最大挑战是如何确保其使用者能够得到最新的制品，避免开发协同出现问题。

配置管理

- IEEE将配置管理定义为[IEEE610.12-1990]：“用技术的和管理的指导和监督方法，来标识和说明配置项的功能和物理特征，控制对这些特征的变更，记录和报告变更处理及其实现状态，并验证与规格需求的一致性”。

配置项

IEEE将配置项定义为[IEEE610.12-1990]：“置于软件配置管理之下的软件配置的各种有关项目，包括各类管理文档、评审记录与文档、软件文档、源码及其可执行码、运行所需的系统软件和支持软件以及有关数据等”。

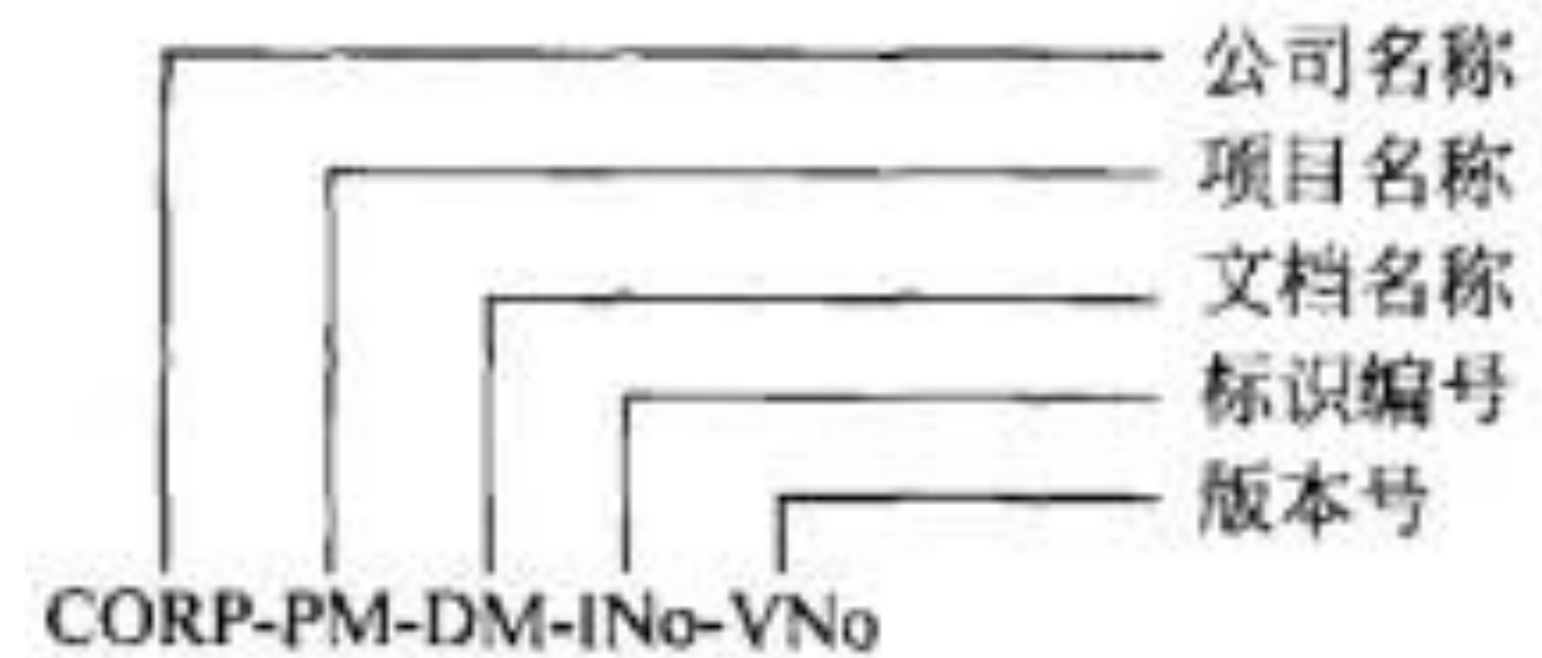
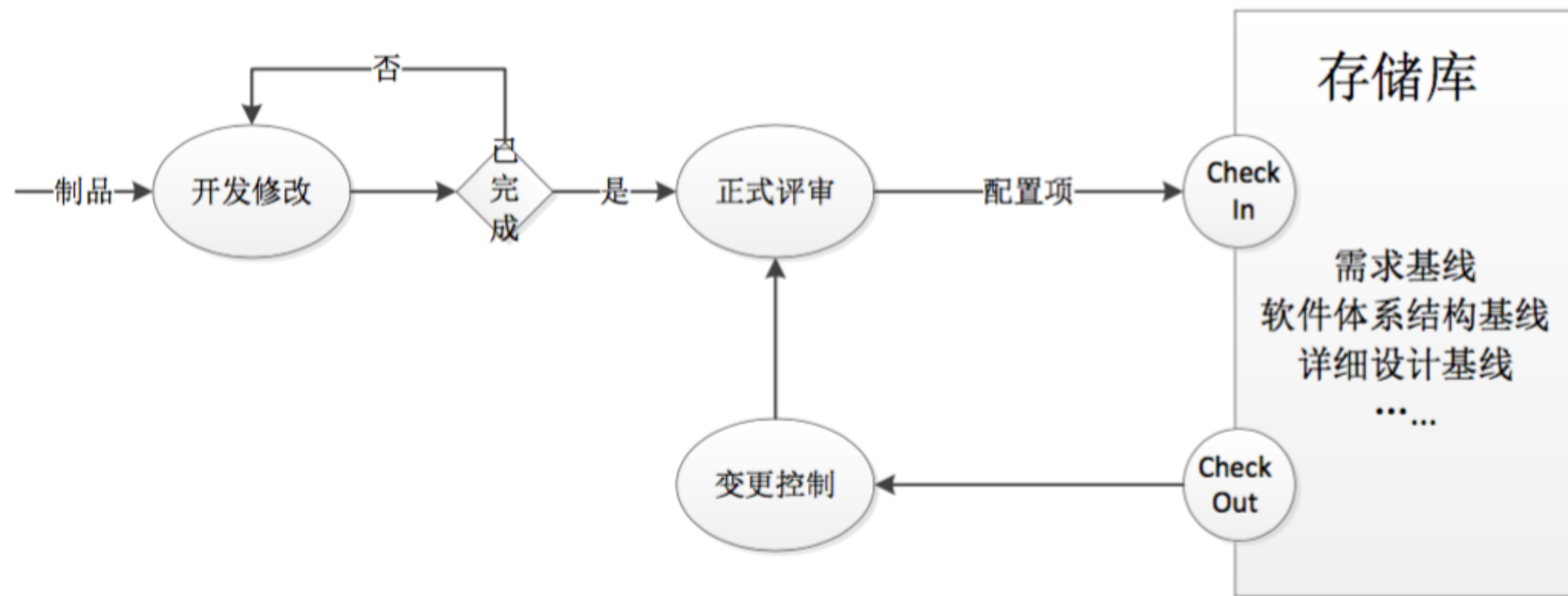


图 1

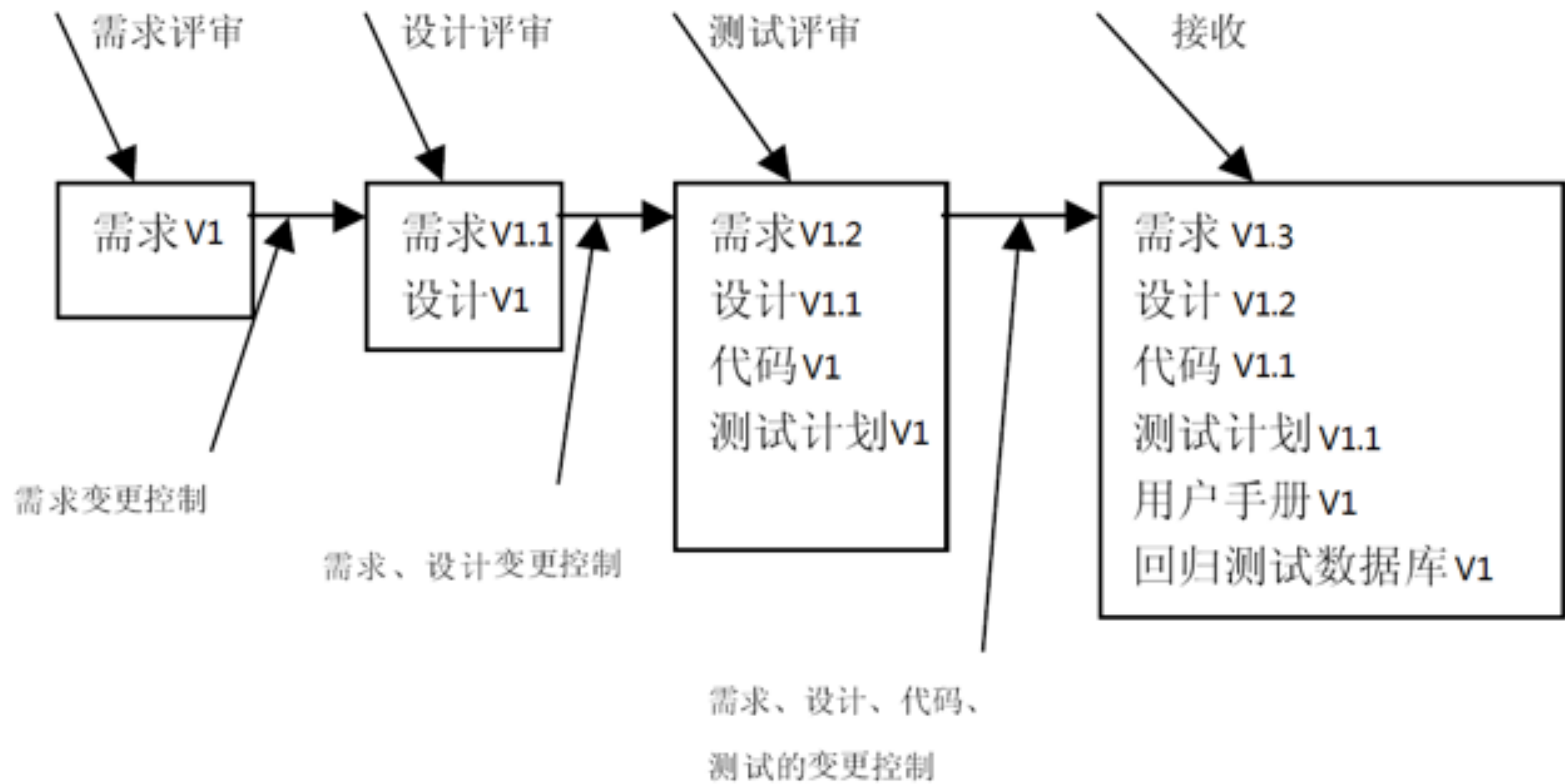
配置标识实例 2



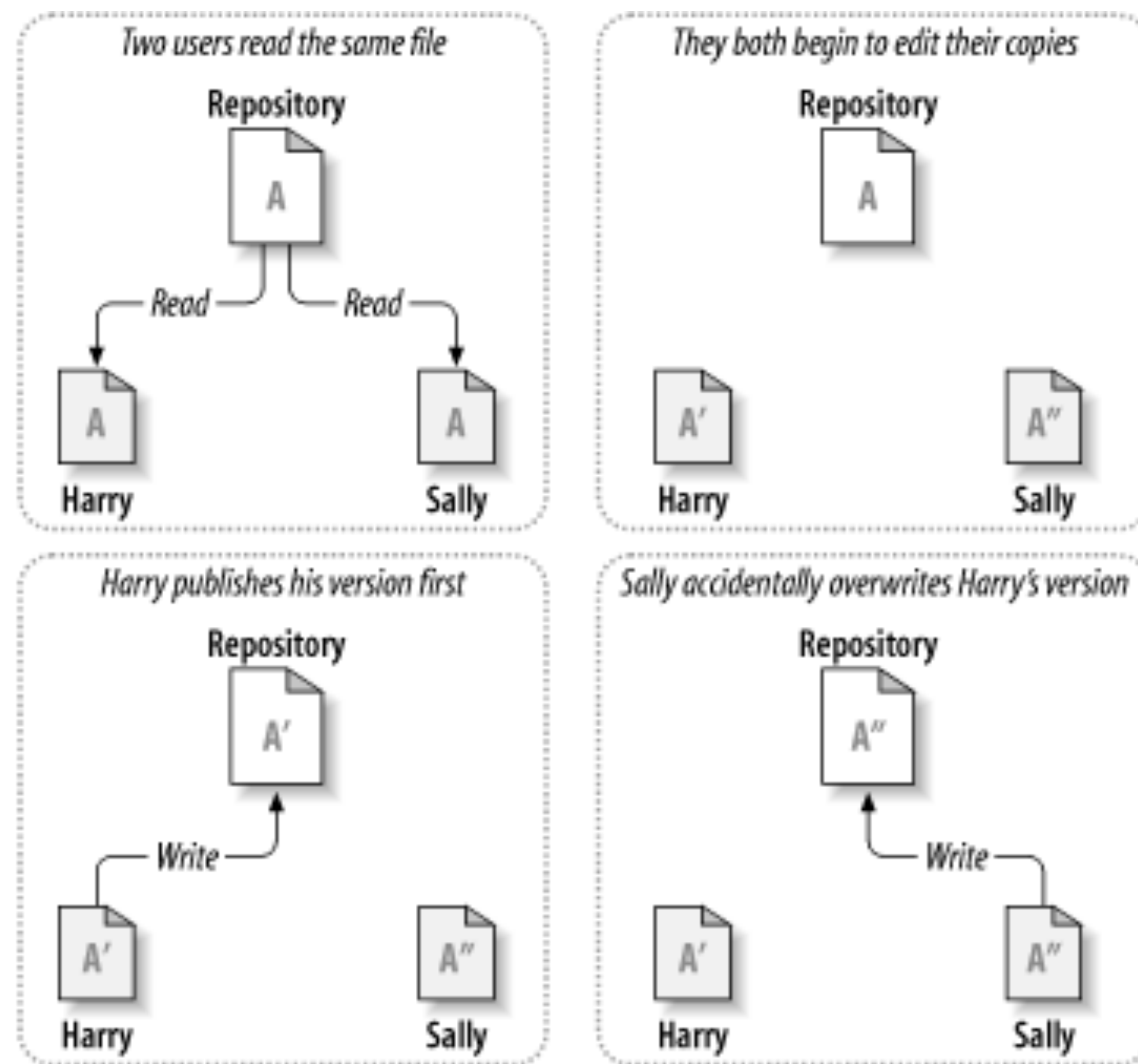
[IEEE610.12-1990]将基线定义为：已经经过正式评审的规格说明或产品，可以作为进一步开发的基础，并且只有通过正式的变更控制过程才能变更。

配置管理活动

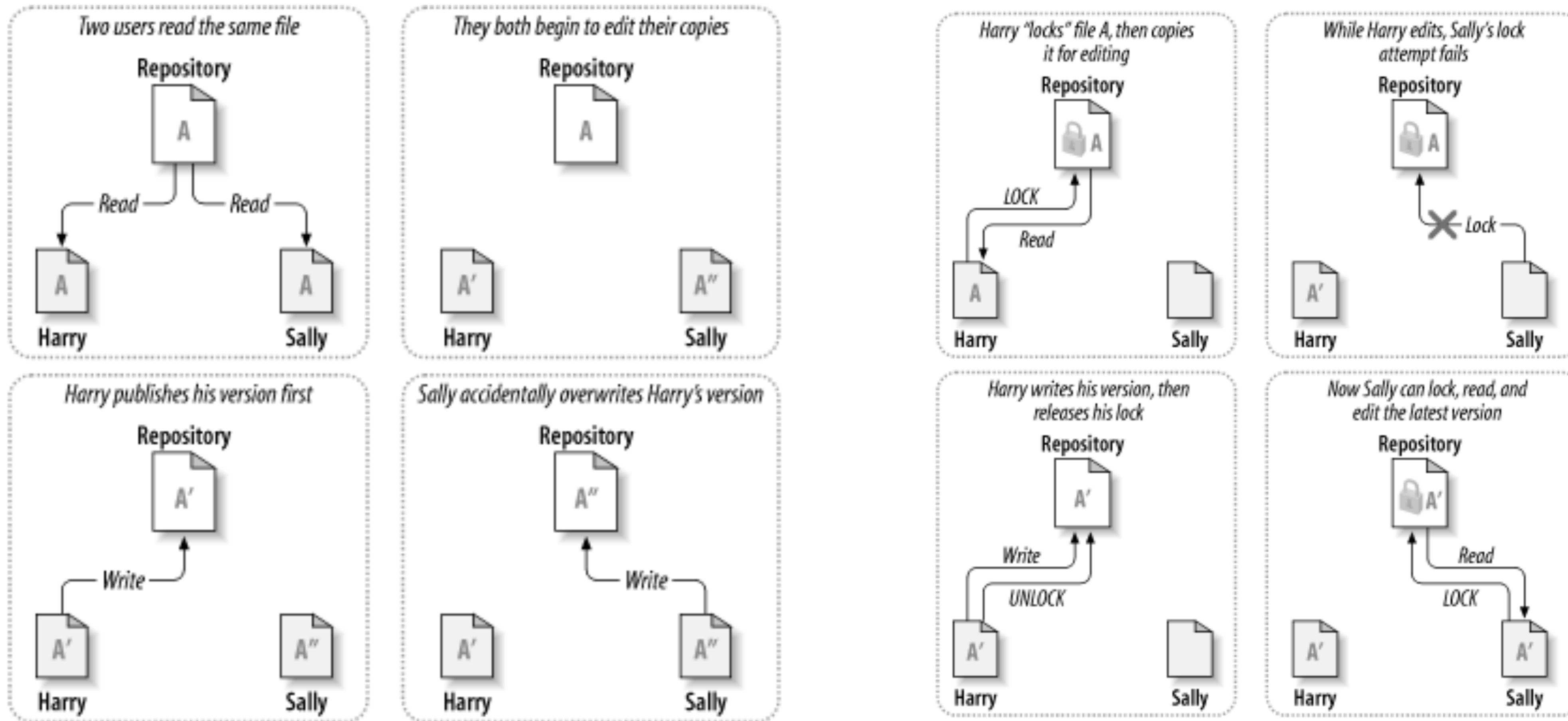
- 标识配置项
- 版本管理
- 变更控制
- 配置审计
- 状态报告
- 软件发布管理



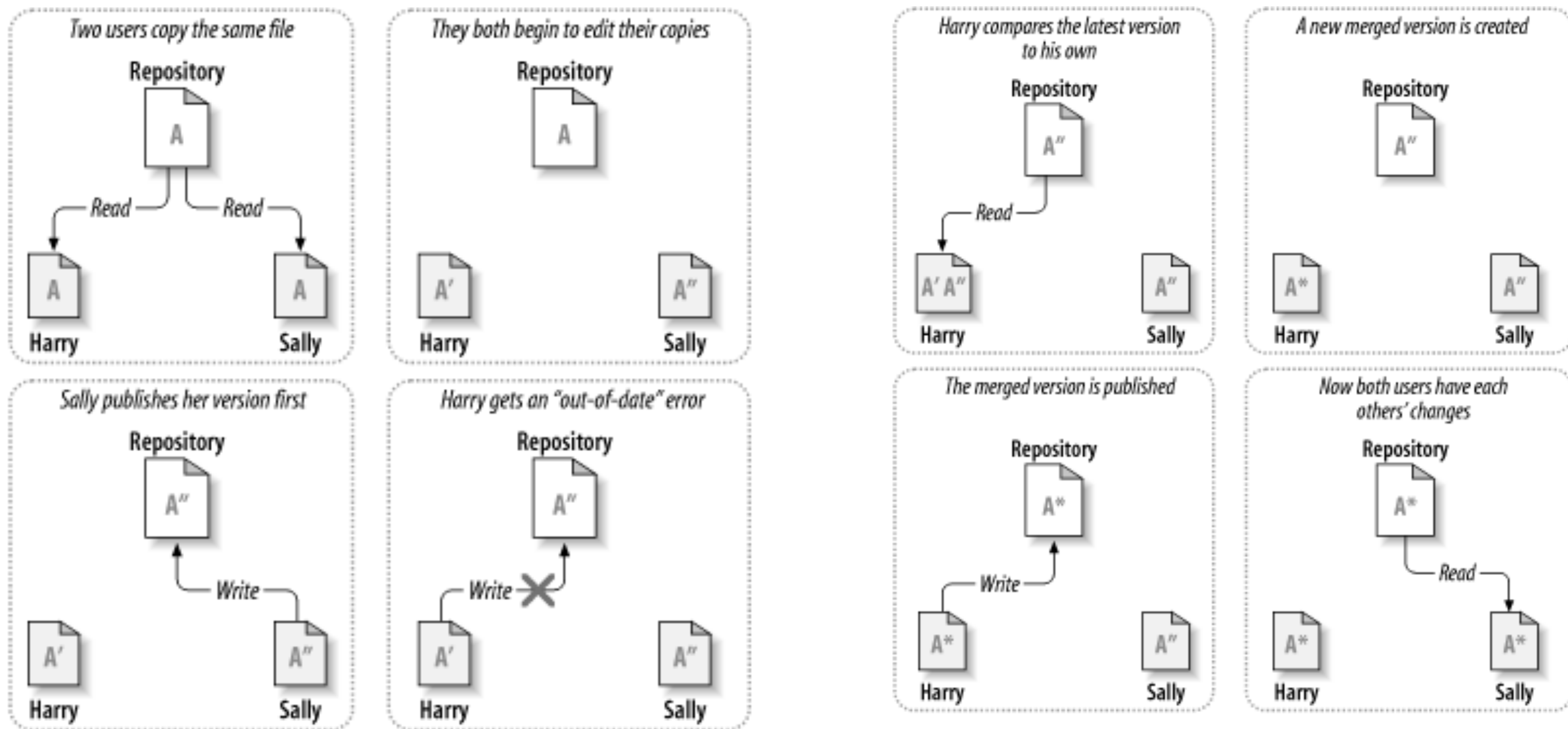
版本管理示意图



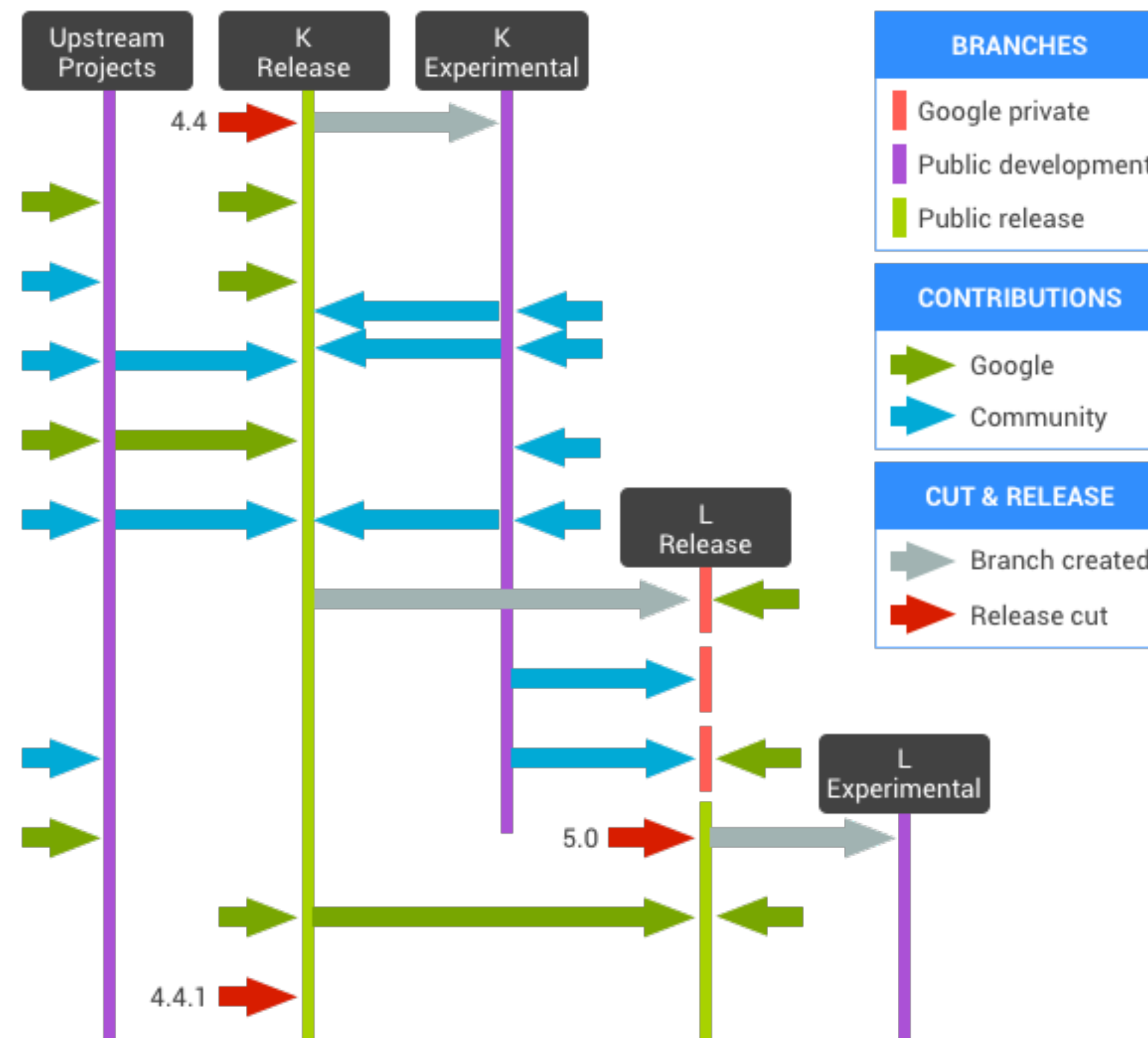
版本控制的问题



lock-modify-unlock solution



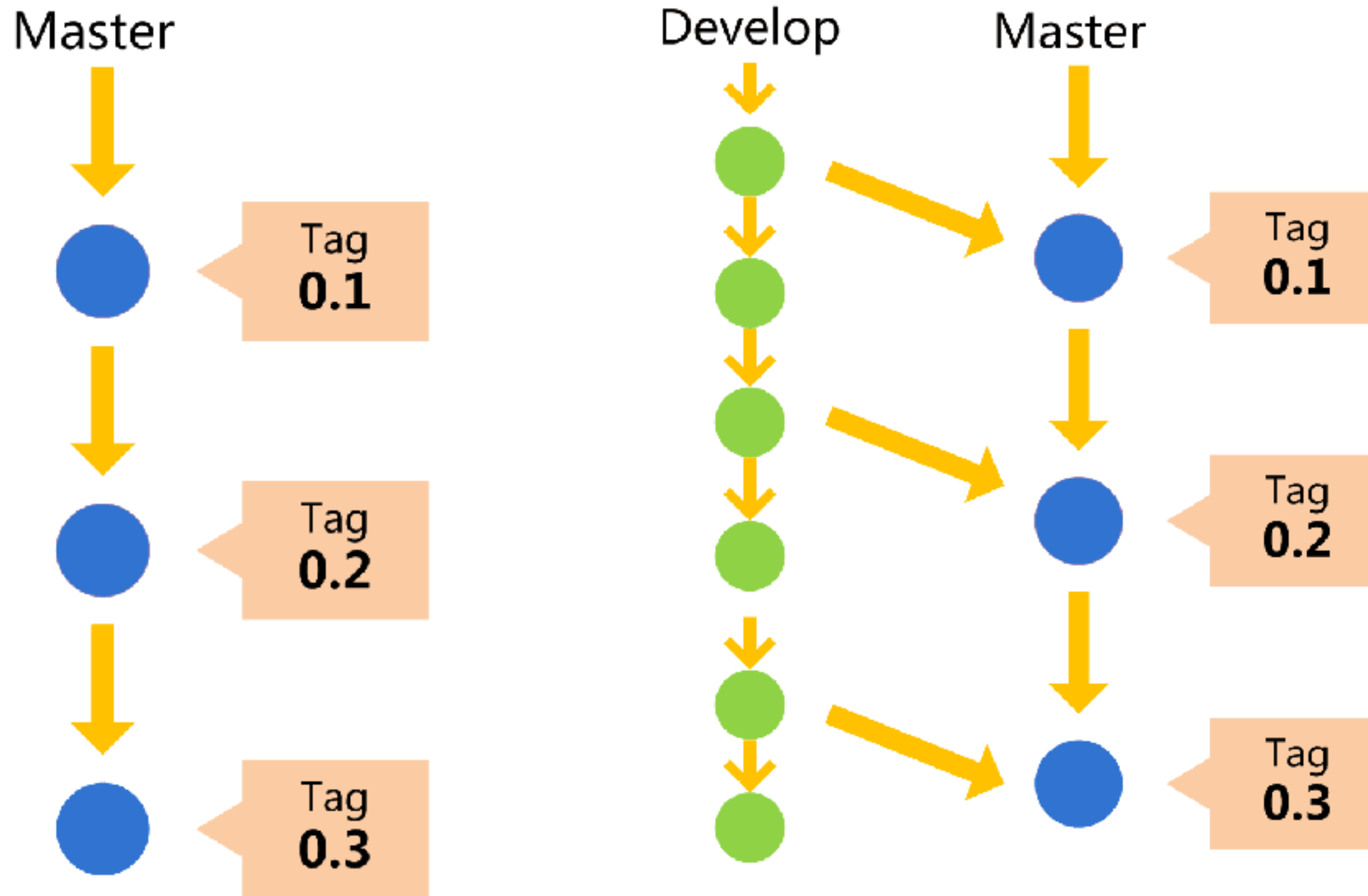
copy-modify-merge solution



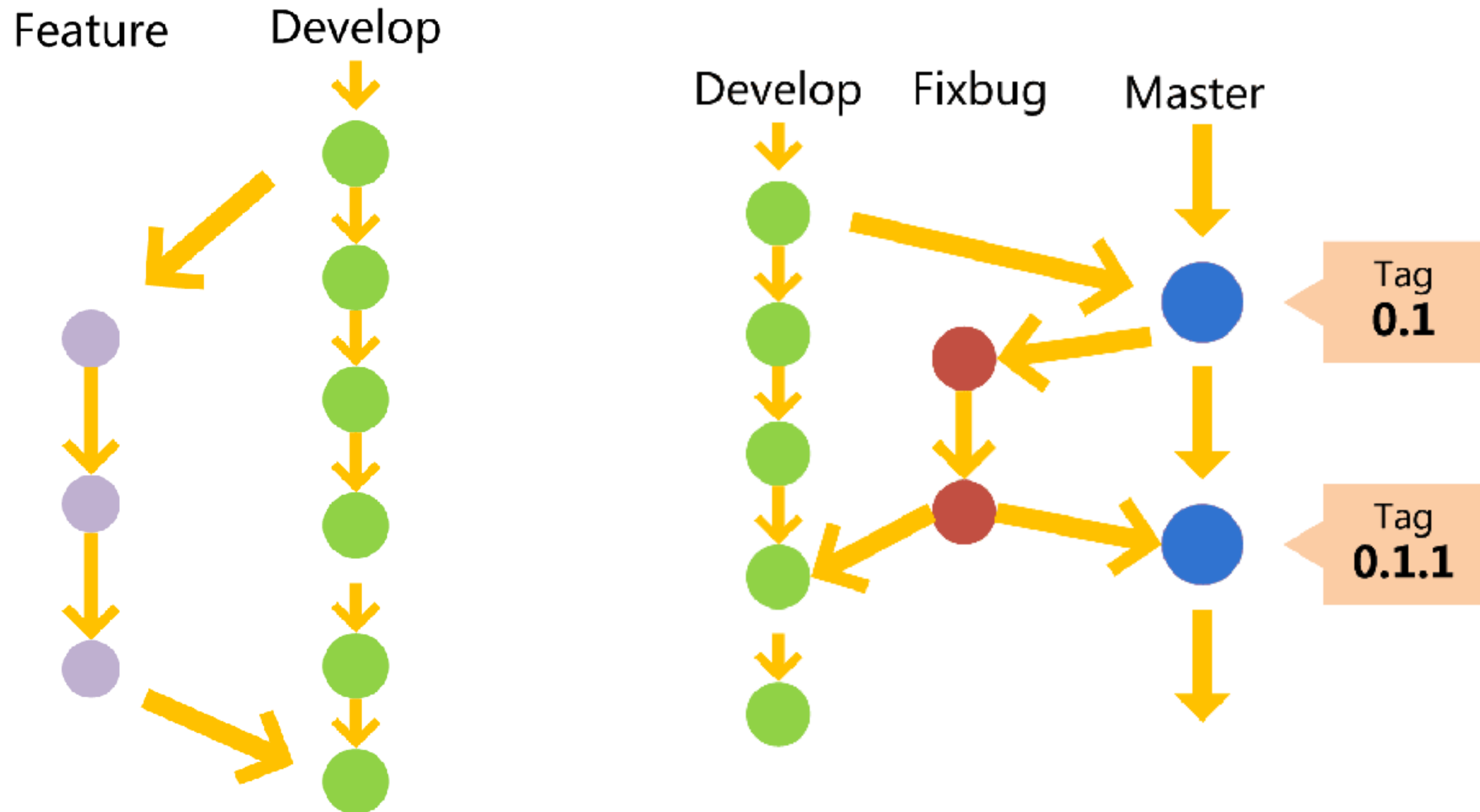
Android Open Source Project Code and Releases

分支管理常见策略

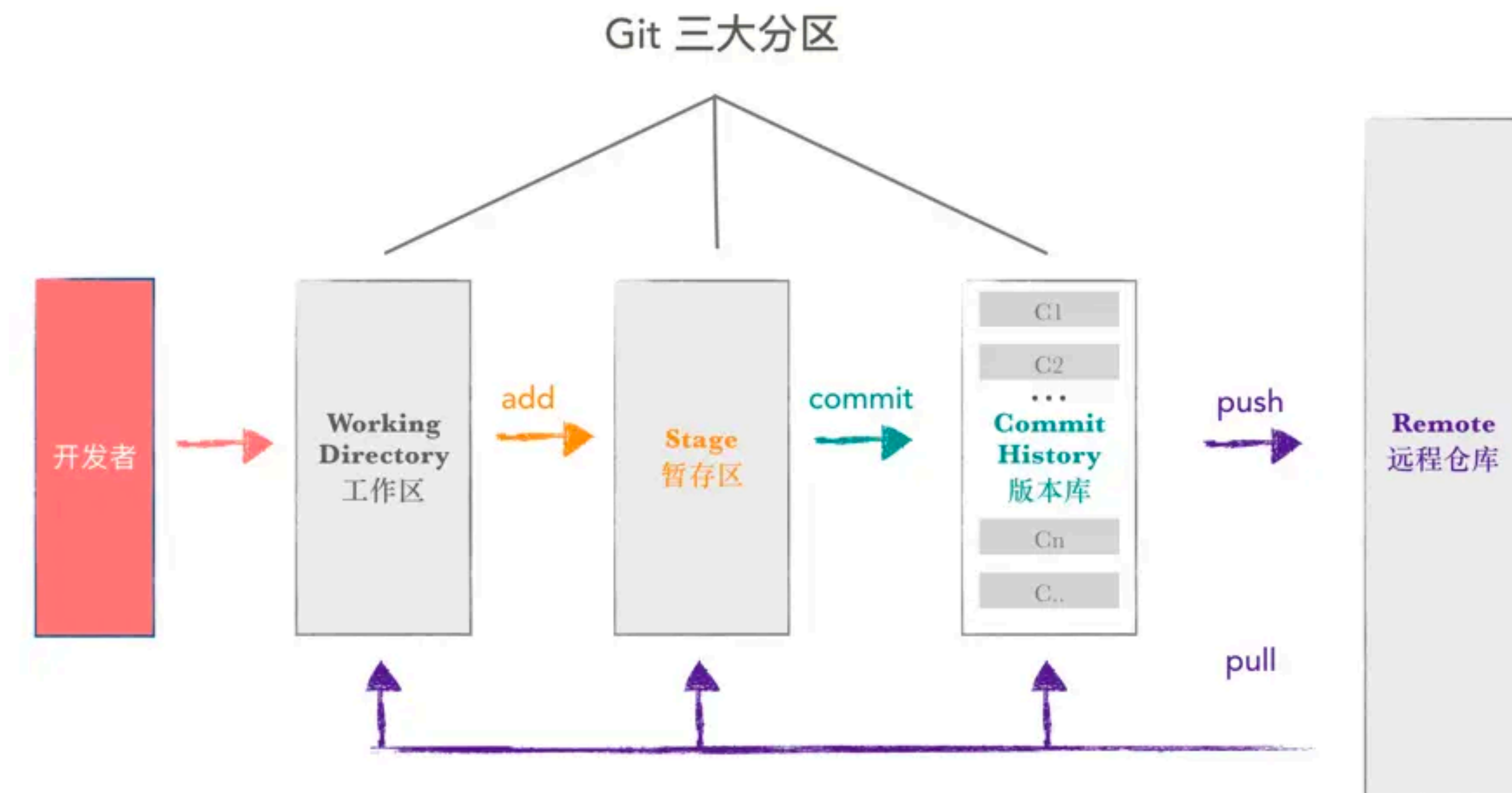
- 主分支 (Master)
- 开发分支 (Develop)
- 临时性分支
 - 功能 (Feature)
 - 预发布 (Release)
 - 修补bug (Fixbug)



Master & Develop



Feature & Fixbug



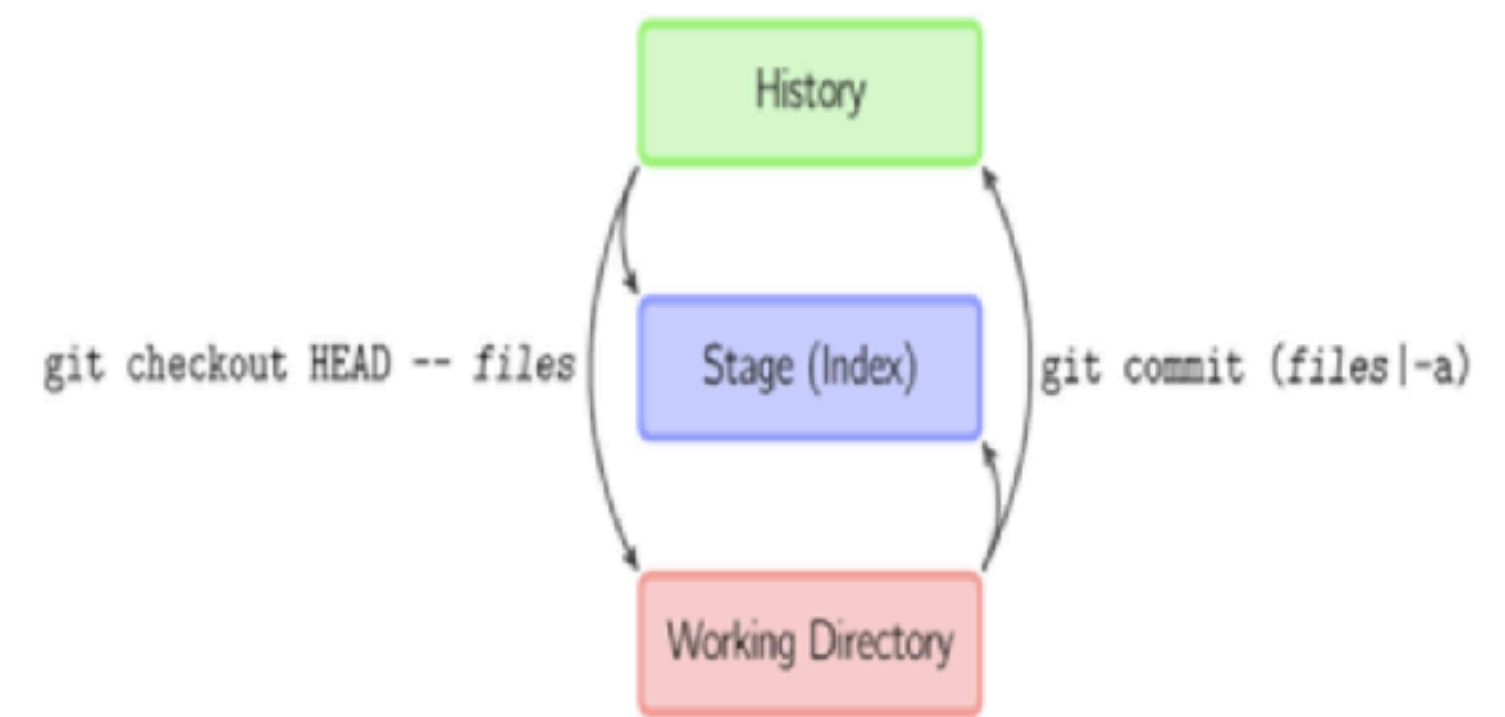
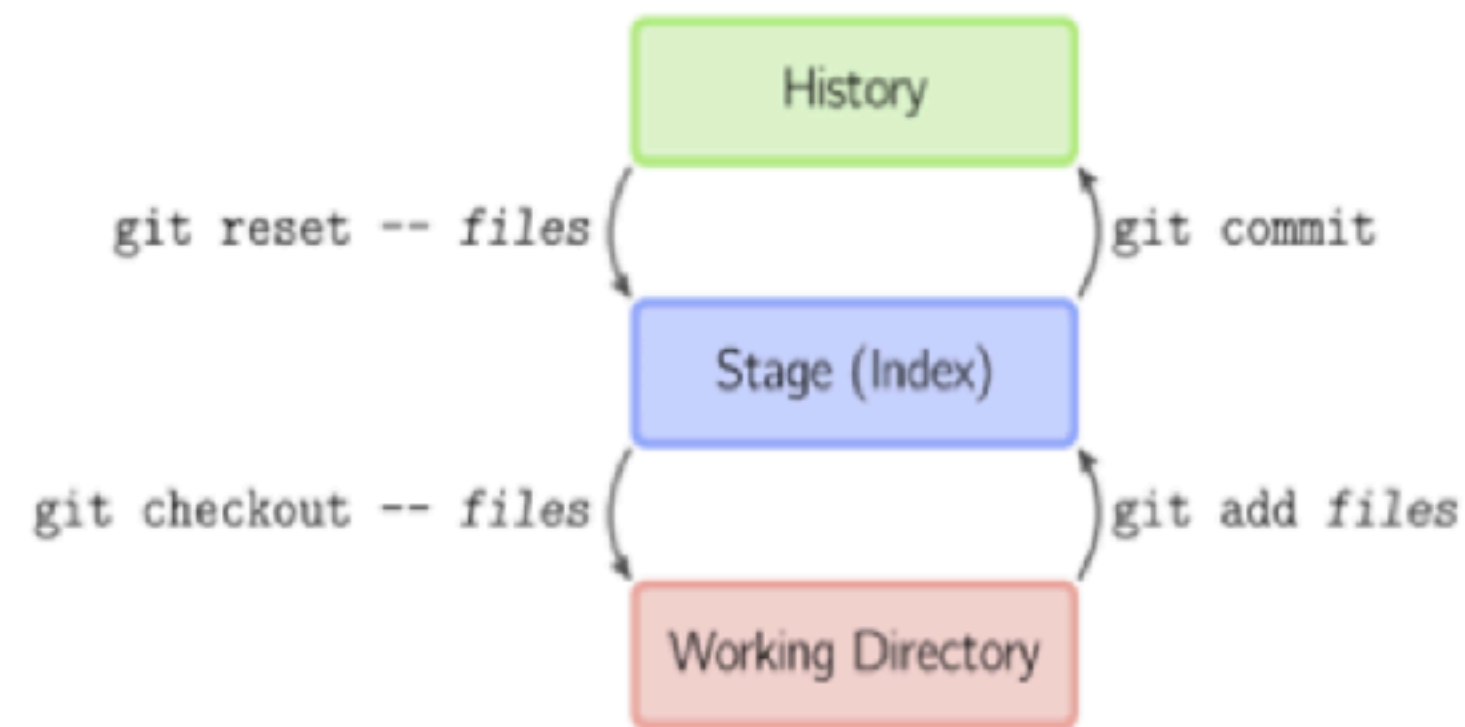
Git的分区

查看不同

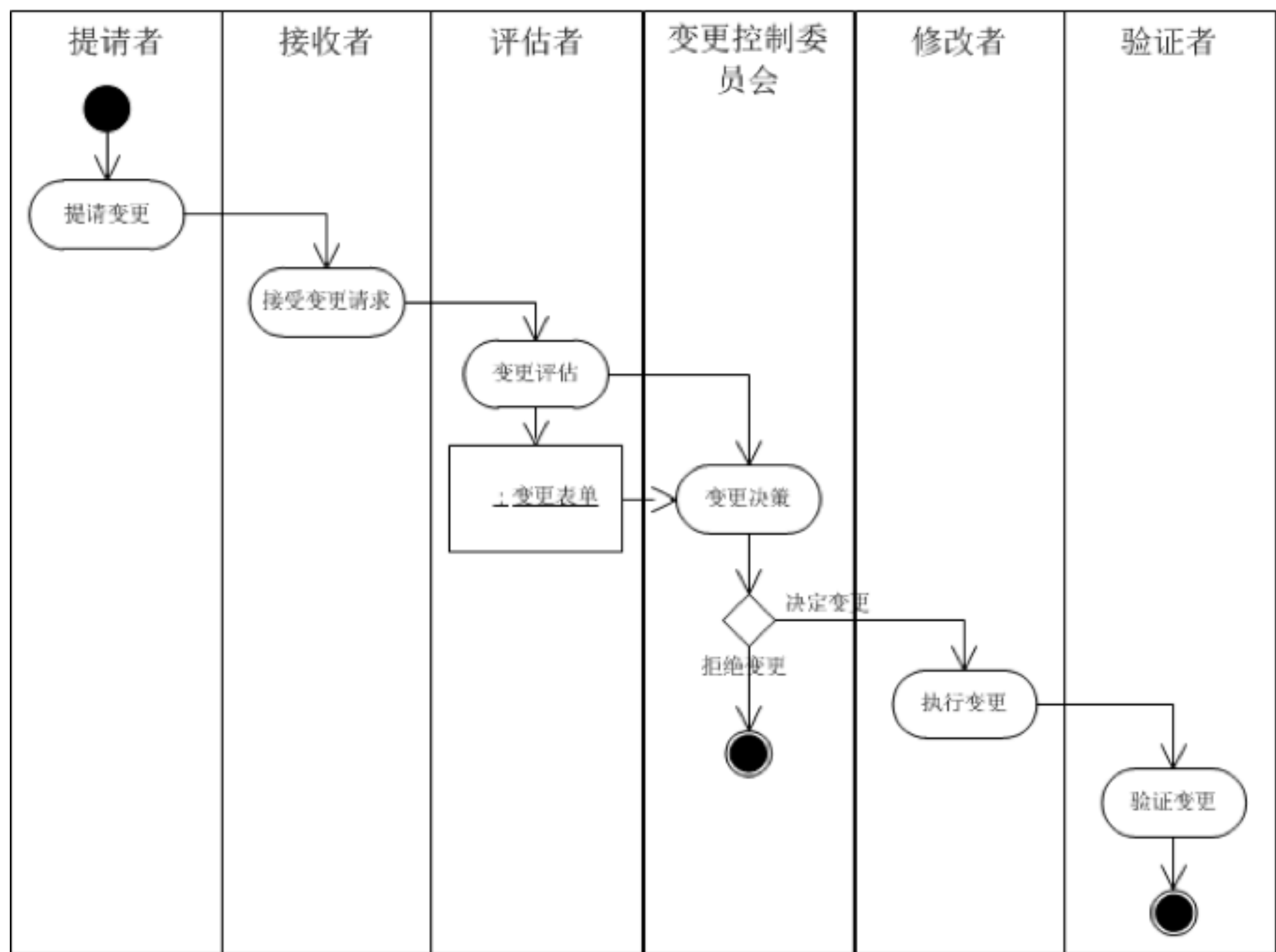
- git diff
 - 工作区 vs 暂存区
- git diff head
 - 工作区 vs 版本库
- git diff --cached
 - 暂存区 vs 版本库

为啥要暂存区

- 增加灵活性
 - 修改了4个文件，在不放弃任何修改的情况下，其中一个文件不想提交，如何操作？（没add : `git add` 已经add: `git reset --soft`）
 - 修改到一半的文件，突然间不需要或者放弃修改了，怎么恢复未修改前文件？（`git checkout`）
 - 代码写一半，被打断去做其他功能开发，未完成代码保存？（`git stash`）
 - 代码写一半，发现忘记切换分支了？（`git stash & git checkout`）
 - 代码需要回滚了？（`git reset`）



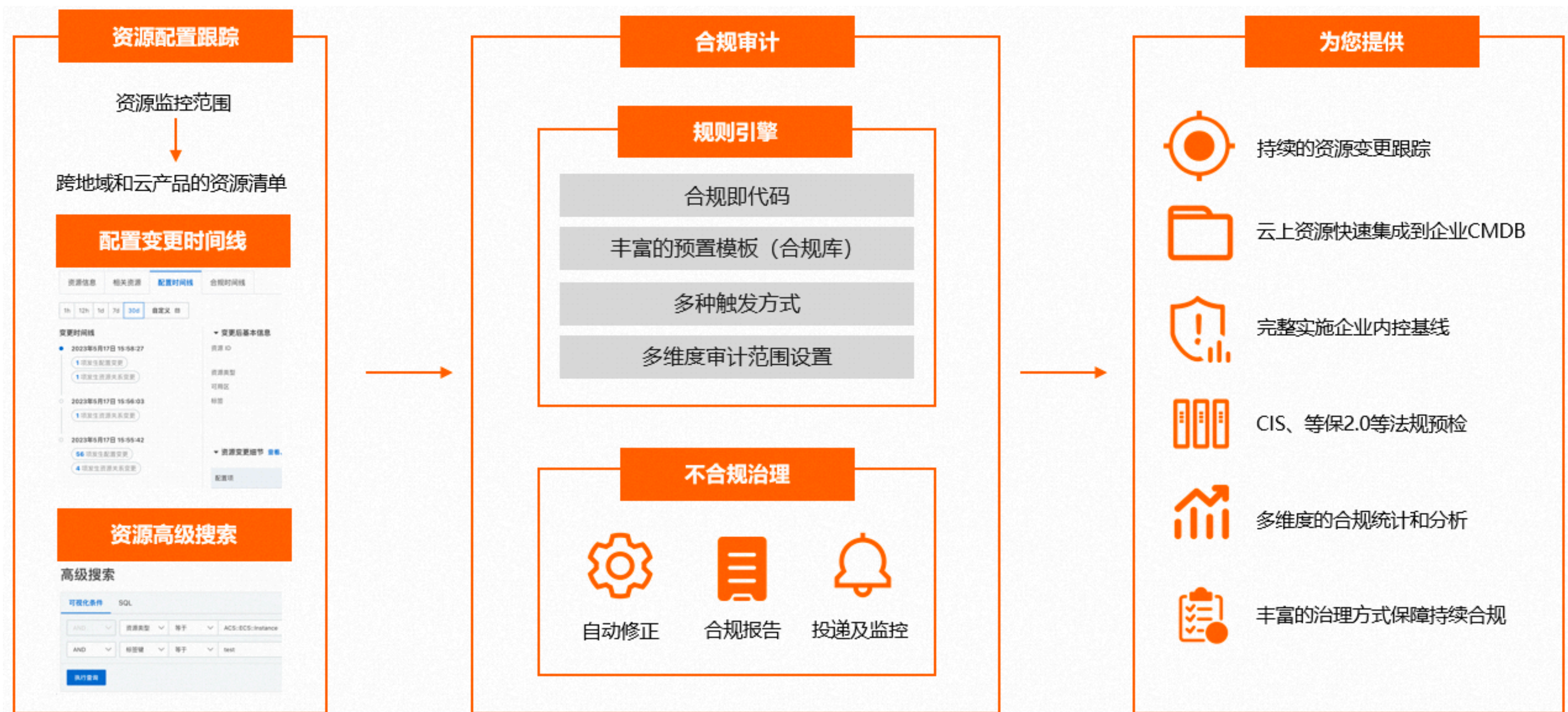
- 作者：段浅浅儿
- 链接：<https://www.jianshu.com/p/f79ae362ef9e>



变更控制

变更请求表单

项目名称:	请求编号:
申请人:	申请日期:
申请理由及优先级:	
变更请求描述:	
评估人:	评估日期:
评估优先级:	变更类型:
影响范围:	
工作量估算:	
变更评价:	
提交CCB日期:	CCB决策日期:
CCB决定:	
修改人:	修改日期:
修改结果:	
验证人:	验证日期:
验证结果:	
备注:	



配置审计 — 阿里云案例

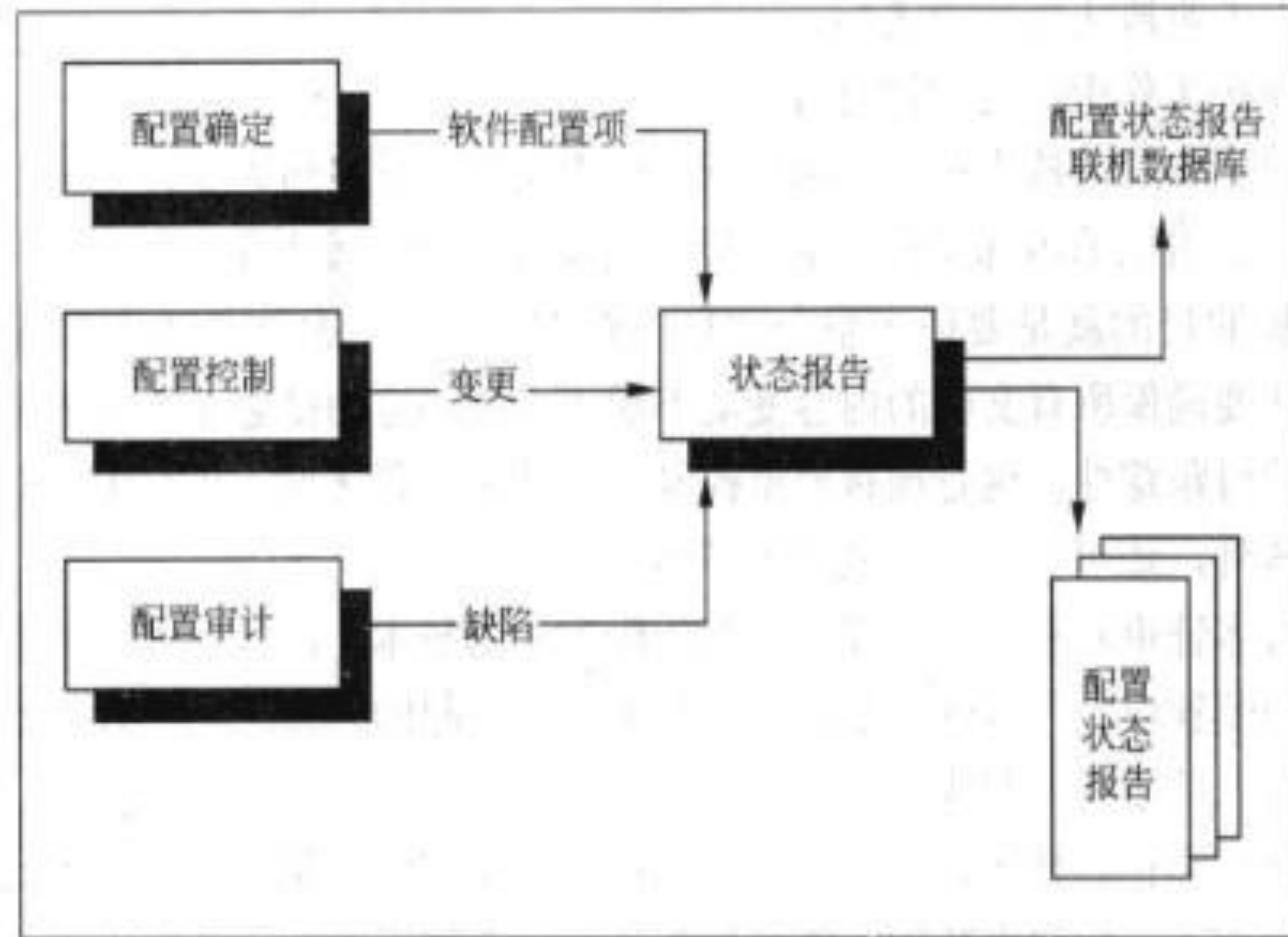


图1 配置状态报告

状态报告

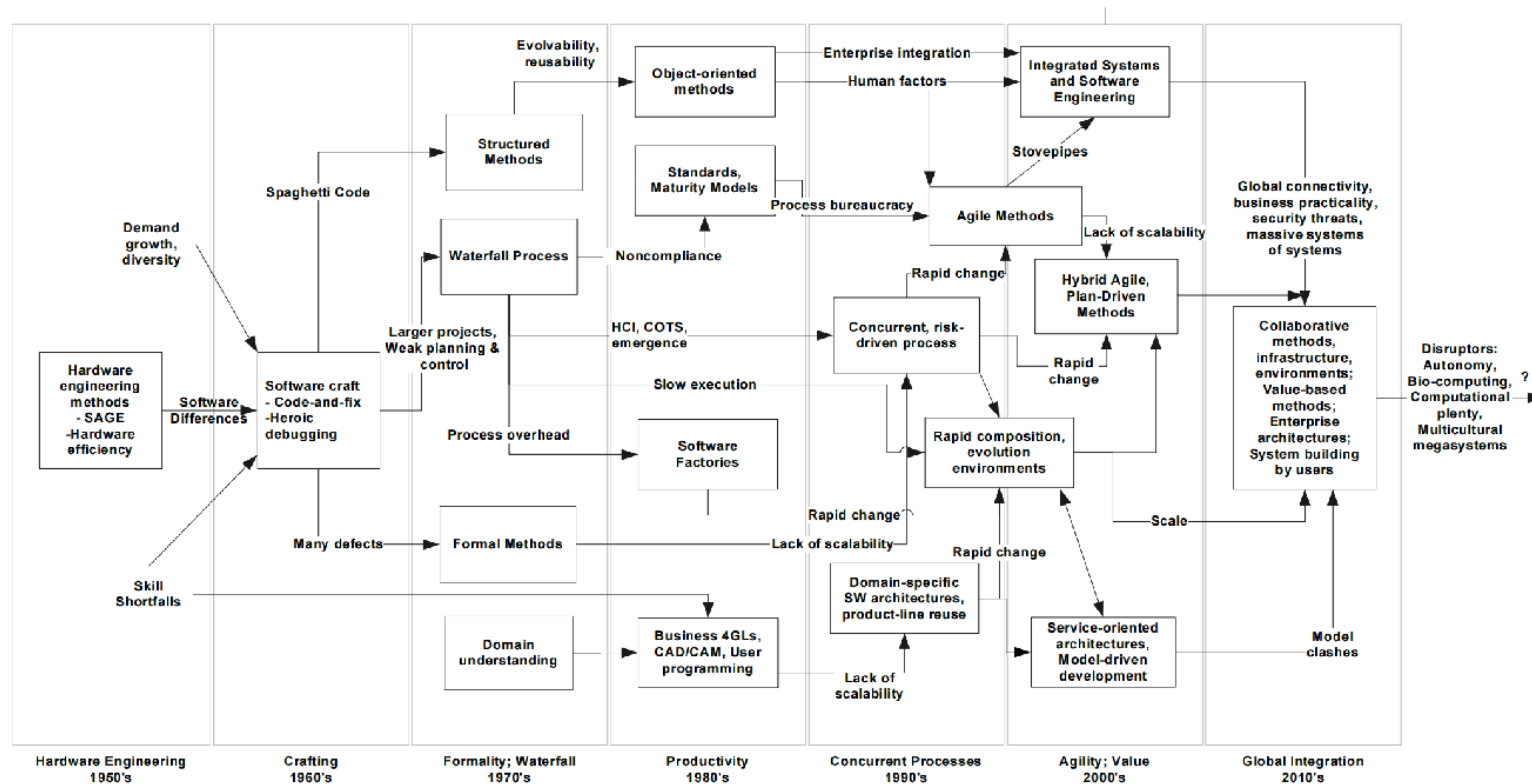
How to learn SE

- “你纵然学得了剑法，倘若使出时剑法不纯，毕竟还是敌不了当世高手，此刻你已经得到了门径，要想多胜少败，再苦练二十年，便可和天下英雄一较长短了”

— 风清扬

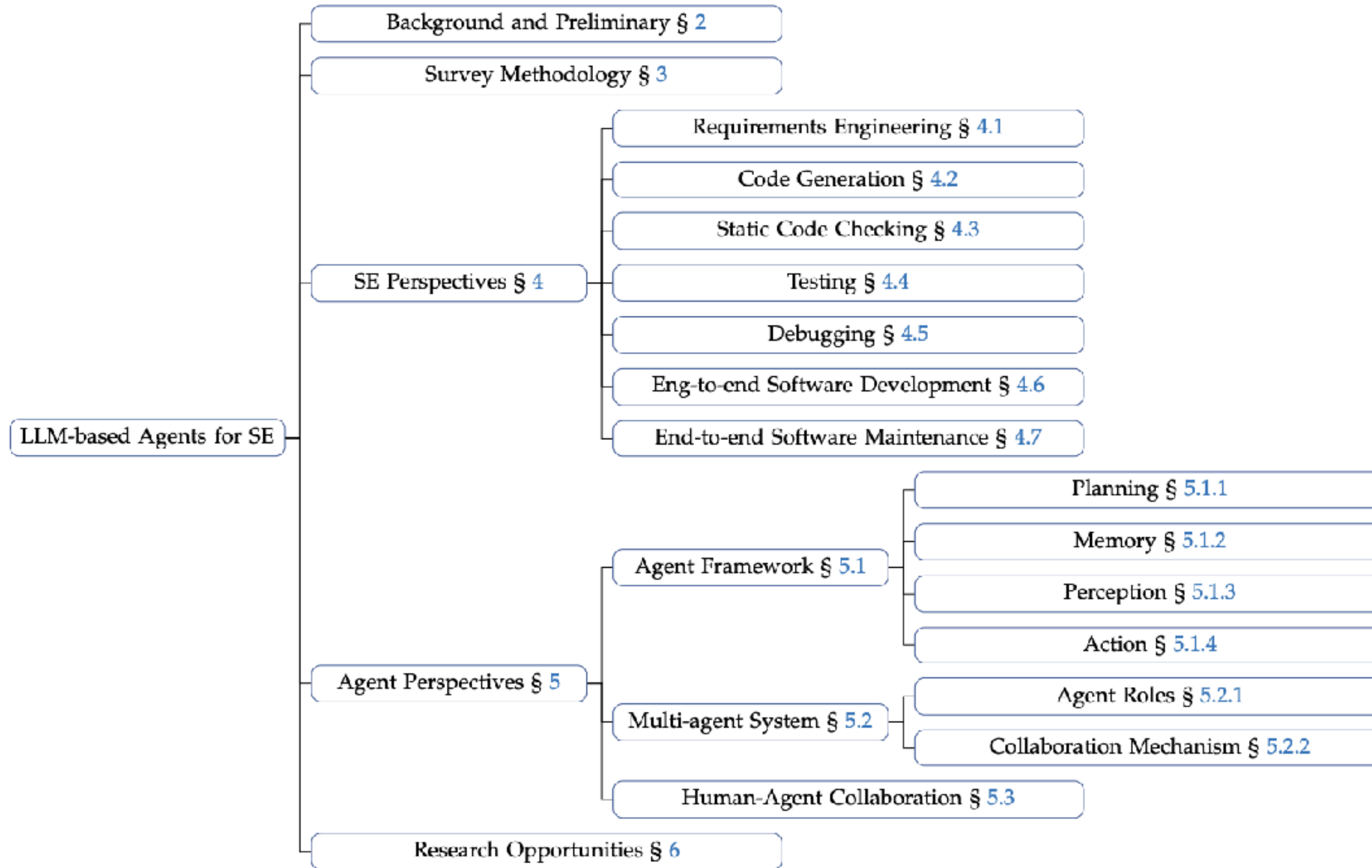
拥抱AI

Paper Reading



A View of 20th and 21st Century Software Engineering

Barry Boehm



Large Language Model-Based Agents for Software Engineering: A Survey

<https://arxiv.org/pdf/2409.02977>